Chapter 4. Threads

This chapter explains the data structures and algorithms that deal with threads and thread scheduling in Windows. The first section shows how to create threads. Then the internals of threads and thread scheduling are described. The chapter concludes with a discussion of thread pools.

Creating threads

Before discussing the internal structures used to manage threads, let's take a look at creating threads from an API perspective to give a sense of the steps and arguments involved.

The simplest creation function in user mode is `CreateThread`. This function creates a thread in the current process, accepting the following arguments:

- **An optional security attributes structure** This specifies the security descriptor to attach to the newly created thread. It also specifies whether the thread handle is to be created as inheritable. (Handle inheritance is discussed in Chapter 8, "System mechanisms," in *Windows Internals Part 2*.)

- **An optional stack size** If zero is specified, a default is taken from the executable's header. This always applies to the first thread in a user-mode process. (Thread's stack is discussed further in **Chapter 5**, "**Memory management**.")

- **A function pointer** This serves as the entry point for the new thread's execution.

■ **An optional argument** This is to pass to the thread's function.

■ **Optional flags** One controls whether the thread starts suspended (`CREATE_SUSPENDED`). The other controls the interpretation of the stack size argument (initial committed size or maximum reserved size).

On successful completion, a non-zero handle is returned for the new thread and, if requested by the caller, the unique thread ID.

An extended thread creation function is `CreateRemoteThread`. This function accepts an extra argument (the first), which is a handle to a target process where the thread is to be created. You can use this function to inject a thread into another process. One common use of this technique is for a debugger to force a break in a debugged process. The debugger injects the thread, which immediately causes a breakpoint by calling the `DebugBreak` function. Another common use of this technique is for one process to obtain internal information about another process, which is easier when running within the target process context (for example, the entire address space is visible). This could be done for legitimate or malicious purposes.

To make `CreateRemoteThread` work, the process handle must have been obtained with enough access rights to allow such operation. As an extreme example, protected processes cannot be injected in this way because handles to such processes can be obtained with very limited rights only.

The final function worth mentioning here is `CreateRemoteThreadEx`, which is a superset of `CreateThread` and `CreateRemoteThread`. In fact, the implementation of `CreateThread` and `CreateRemoteThread` simply calls `CreateRemoteThreadEx` with the appropriate defaults. `CreateRemoteThreadEx` adds the ability to provide an attribute list (similar to the `STARTUPINFOEX` structure's role with an additional member over `STARTUPINFO` when creating processes). Examples of attributes include setting the ideal processor and group affinity (both discussed later in this chapter).

If all goes well, `CreateRemoteThreadEx` eventually calls `NtCreateThreadEx` in Ntdll.dll. This makes the usual transition to kernel mode, where execution continues in the executive function `NtCreateThreadEx`. There, the kernel mode part of thread creation occurs (described later in this chapter, in the "**Birth of a thread**" section).

Creating a thread in kernel mode is achieved with the `PsCreateSystemThread` function (documented in the WDK). This is useful for drivers that need independent work to be processes within the system process (meaning it's not associated with any particular process). Technically, the function can be used to create a thread under any process, which is less useful for drivers.

Exiting a kernel thread's function does not automatically destroy the thread object. Instead, drivers must call `PsTerminateSystemThread` from within the thread function to properly terminate the thread. Consequently, this function never returns.

Thread internals

This section discusses the internal structures used within the kernel (and some in user mode) to manage a thread. Unless explicitly stated otherwise, you can assume that anything in this section applies to both user-mode threads and kernel-mode system threads.

Data structures

At the operating-system (OS) level, a Windows thread is represented by an executive thread object. The executive thread object encapsulates an `ETHREAD` structure, which in turn contains a `KTHREAD` structure as its first member. These are illustrated in **Figure 4-1** (`ETHREAD`) and **Figure 4-2** (`KTHREAD`). The `ETHREAD` structure and the other structures it points to exist in the system address space. The only exception is the thread environment block (TEB), which exists in the process address space (similar to a PEB, because user-mode components need to access it).

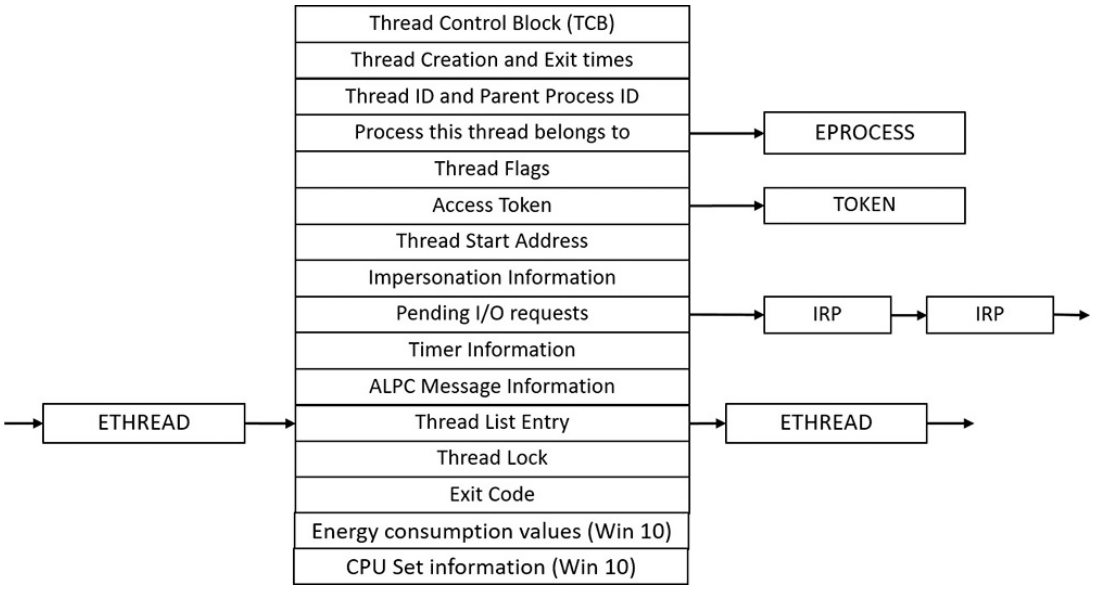


FIGURE 4-1 Important fields of the executive thread structure (ETHREAD).

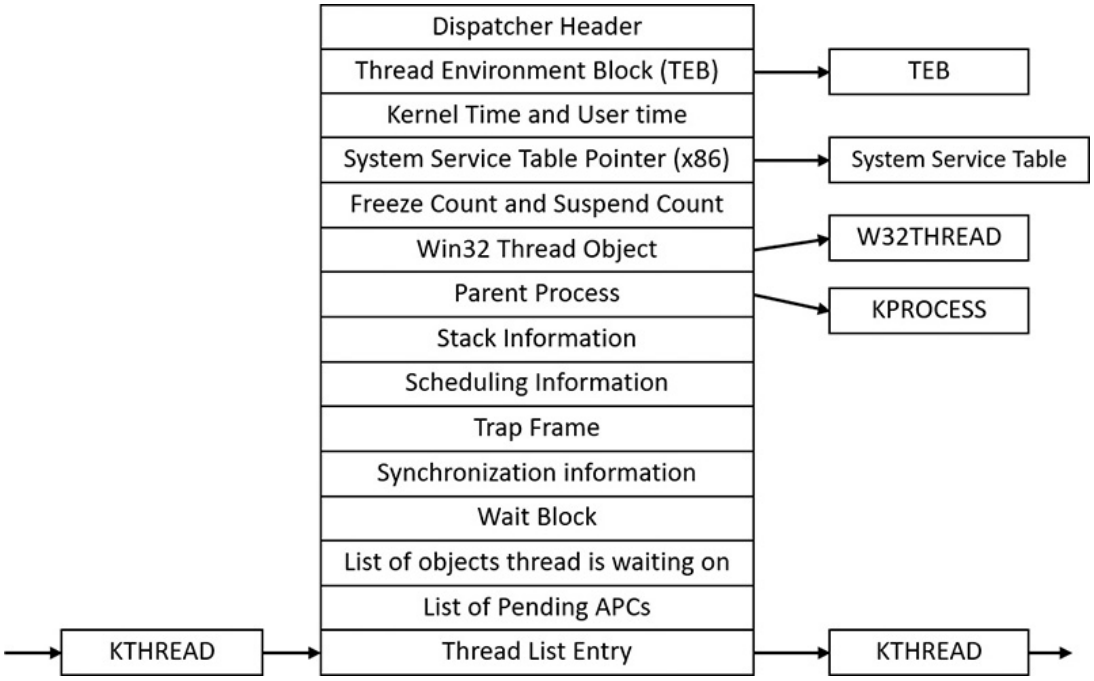


FIGURE 4-2 Important fields of the kernel thread structure (KTHREAD).

The Windows subsystem process (Csrss) maintains a parallel structure for each thread created in a Windows subsystem application, called the `CSR_THREAD`. For threads that have called a Windows subsystem `USER` or `GDI` function, the kernel-mode portion of the Windows subsystem (Win32k.sys) maintains a per-thread data structure (`W32THREAD`) that the `KTHREAD` structure points to.

NOTE

The fact that the executive, high-level, graphics-related, Win32k thread structure is pointed to by `KTHREAD` instead of the `ETHREAD` appears to be a layer violation or oversight in the standard kernel's abstraction architecture. The scheduler and other low-level components do not use this field.

Most of the fields illustrated in **Figure 4-1** are self-explanatory. The first member of the `ETHREAD` is called `Tcb`. This is short for *thread control block*, which is a structure of type `KTHREAD`. Following that are the thread identification information, the process identification information (including a pointer to the owning process so that its environment information can be accessed), security information in the form of a pointer to the access token and impersonation information, fields relating to Asynchronous Local Procedure Call (ALPC) messages, pending I/O requests (IRPs) and Windows 10–specific fields related to power management (described in **Chapter 6**, "**I/O system**") and CPU Sets (described later in this chapter). Some of these key fields are covered in more detail elsewhere in this book. For more details on the internal structure of an `ETHREAD` structure, you can use the kernel debugger `dt` command to display its format.

Let's take a closer look at two of the key thread data structures referred to in the preceding text: `ETHREAD` and `KTHREAD`. The `KTHREAD` structure (which is the `Tcb` member of the `ETHREAD`) contains information that the Windows kernel needs to perform thread scheduling, synchronization, and timekeeping functions.

EXPERIMENT: DISPLAYING ETHREAD AND KTHREAD STRUCTURES

You can display the `ETHREAD` and `KTHREAD` structures with the `dt` command in the kernel debugger. The following output shows the format of an `ETHREAD` on a 64-bit Windows 10 system:

Click here to view code image

```
lkd> dt nt!_ethread
   +0x000 Tcb            : _KTHREAD
   +0x5d8 CreateTime     : _LARGE_INTEGER
   +0x5e0 ExitTime       : _LARGE_INTEGER
...
   +0x7a0 EnergyValues   : Ptr64 _THREAD_ENERGY_VALUES
   +0x7a8 CmCellReferences : Uint4B
   +0x7b0 SelectedCpuSets  : Uint8B
   +0x7b0 SelectedCpuSetsIndirect : Ptr64 Uint8B
   +0x7b8 Silo           : Ptr64 _EJOB
```

You can display the `KTHREAD` with a similar command or by typing `dt nt!_ETHREAD Tcb`, as shown in the experiment "Displaying the format of an EPROCESS structure" in **Chapter 3**, "**Processes and jobs**."

Click here to view code image

```
lkd> dt nt!_kthread
   +0x000 Header         : _DISPATCHER_HEADER
   +0x018 SListFaultAddress : Ptr64 Void
   +0x020 QuantumTarget  : Uint8B
   +0x028 InitialStack   : Ptr64 Void
   +0x030 StackLimit     : Ptr64 Void
   +0x038 StackBase      : Ptr64 Void
   +0x040 ThreadLock     : Uint8B
   +0x048 CycleTime      : Uint8B
   +0x050 CurrentRunTime : Uint4B
...
   +0x5a0 ReadOperationCount : Int8B
   +0x5a8 WriteOperationCount : Int8B
```

+0x5b0 OtherOperationCount : Int8B
+0x5b8 ReadTransferCount : Int8B
+0x5c0 WriteTransferCount : Int8B
+0x5c8 OtherTransferCount : Int8B
+0x5d0 QueuedScb : Ptr64 _KSCB

EXPERIMENT: USING THE KERNEL DEBUGGER !THREAD COMMAND

The kernel debugger `!thread` command dumps a subset of the information in the thread data structures. Some key elements of the information the kernel debugger displays can't be displayed by any utility, including the following information:

- Internal structure addresses
- Priority details
- Stack information
- The pending I/O request list
- For threads in a wait state, the list of objects the thread is waiting for

To display thread information, use either the `!process` command (which displays all the threads of a process after displaying the process information) or the `!thread` command with the address of a thread object to display a specific thread.

Let's find all instances of explorer.exe:

Click here to view code image

```
lkd> !process 0 0 explorer.exe
PROCESS ffffe00017f3e7c0
    SessionId: 1  Cid: 0b7c    Peb: 00291000  ParentCid: 0c34
    DirBase: 19b264000  ObjectTable: ffffc00007268cc0  HandleCount:
2248.
```

```
    Image: explorer.exe

PROCESS ffffe00018c817c0
    SessionId: 1  Cid: 23b0    Peb: 00256000  ParentCid: 03f0
    DirBase: 2d4010000  ObjectTable: ffffc0001aef0480  HandleCount:
2208.
    Image: explorer.exe
```

We'll select one of the instances and show its threads:

Click here to view code image

```
lkd> !process ffffe00018c817c0 2
PROCESS ffffe00018c817c0
    SessionId: 1  Cid: 23b0    Peb: 00256000  ParentCid: 03f0
    DirBase: 2d4010000  ObjectTable: ffffc0001aef0480  HandleCount:
2232.
    Image: explorer.exe

        THREAD ffffe0001ac3c080  Cid 23b0.2b88  Teb: 0000000000257000
Win32Thread:
ffffe0001570ca20 WAIT: (UserRequest) UserMode Non-Alertable
            ffffe0001b6eb470  SynchronizationEvent

        THREAD ffffe0001af10800  Cid 23b0.2f40  Teb: 0000000000265000
Win32Thread:
ffffe000156688a0 WAIT: (UserRequest) UserMode Non-Alertable
            ffffe000172ad4f0  SynchronizationEvent
            ffffe0001ac26420  SynchronizationEvent

        THREAD ffffe0001b69a080  Cid 23b0.2f4c  Teb: 0000000000267000
Win32Thread:
ffffe000192c5350 WAIT: (UserRequest) UserMode Non-Alertable
            ffffe00018d83c00  SynchronizationEvent
            ffffe0001552ff40  SynchronizationEvent

...
```

```
    THREAD ffffe00023422080  Cid 23b0.3d8c  Teb: 00000000003cf000
Win32Thread:
ffffe0001eccd790 WAIT: (WrQueue) UserMode Alertable
        ffffe0001aec9080  QueueObject

    THREAD ffffe00023f23080  Cid 23b0.3af8  Teb: 00000000003d1000
Win32Thread:
0000000000000000 WAIT: (WrQueue) UserMode Alertable
        ffffe0001aec9080  QueueObject

    THREAD ffffe000230bf800  Cid 23b0.2d6c  Teb: 00000000003d3000
Win32Thread:
0000000000000000 WAIT: (WrQueue) UserMode Alertable
        ffffe0001aec9080  QueueObject

    THREAD ffffe0001f0b5800  Cid 23b0.3398  Teb: 00000000003e3000
Win32Thread:
0000000000000000 WAIT: (UserRequest) UserMode Alertable
        ffffe0001d19d790  SynchronizationEvent
        ffffe00022b42660  SynchronizationTimer
```

The list of threads is truncated for the sake of space. Each thread shows its address (`ETHREAD`), which can be passed to the `!thread` command; its client ID (`Cid`)–process ID and thread ID (the process ID for all the preceding threads is the same, as they are part of the same explorer.exe process); the Thread Environment Block (TEB, discussed momentarily); and the thread state (most should be in the `Wait` state, with the reason for the wait in parentheses). The next line may show a list of synchronization objects the threads is waiting on.

To get more information on a specific thread, pass its address to the `!thread` command:

Click here to view code image

```
lkd> !thread ffffe0001d45d800
THREAD ffffe0001d45d800  Cid 23b0.452c  Teb: 000000000026d000
Win32Thread:
ffffe0001aace630 WAIT: (UserRequest) UserMode Non-Alertable
    ffffe00023678350  NotificationEvent
    ffffe00022aeb370  Semaphore Limit 0xffff
    ffffe000225645b0  SynchronizationEvent
Not impersonating
DeviceMap                 ffffc00004f7ddb0
Owning Process            ffffe00018c817c0       Image:         explorer.exe
Attached Process          N/A            Image:         N/A
Wait Start TickCount      7233205        Ticks: 270 (0:00:00:04.218)
Context Switch Count      6570           IdealProcessor: 7
UserTime                  00:00:00.078
KernelTime                00:00:00.046
Win32 Start Address 0c
Stack Init ffffd000271d4c90 Current ffffd000271d3f80
Base ffffd000271d5000 Limit ffffd000271cf000 Call 0000000000000000
Priority 9 BasePriority 8 PriorityDecrement 0 IoPriority 2 PagePriority 5
GetContextState failed, 0x80004001
Unable to get current machine context, HRESULT 0x80004001
Child-SP          RetAddr           : Args to Child                   : Call Site
ffffd000'271d3fc0 fffff803'bef086ca : 00000000'00000000
00000000'00000001
00000000'00000000 00000000'00000000 : nt!KiSwapContext+0x76
ffffd000'271d4100 fffff803'bef08159 : ffffe000'1d45d800
fffff803'00000000
ffffe000'1aec9080 00000000'0000000f : nt!KiSwapThread+0x15a
ffffd000'271d41b0 fffff803'bef09cfe : 00000000'00000000
00000000'00000000
ffffe000'0000000f 00000000'00000003 : nt!KiCommitThreadWait+0x149
ffffd000'271d4240 fffff803'bf2a445d : ffffd000'00000003
ffffd000'271d43c0
00000000'00000000 fffff960'00000006 :
```

nt!KeWaitForMultipleObjects+0x24e
ffffd000'271d4300 fffff803'bf2fa246 : fffff803'bf1a6b40
ffffd000'271d4810
ffffd000'271d4858 ffffe000'20aeca60 :
nt!ObWaitForMultipleObjects+0x2bd
ffffd000'271d4810 fffff803'befdefa3 : 00000000'00000fa0
fffff803'bef02aad
ffffe000'1d45d800 00000000'1e22f198 :
nt!NtWaitForMultipleObjects+0xf6
ffffd000'271d4a90 00007ffe'f42b5c24 : 00000000'00000000
00000000'00000000
00000000'00000000 00000000'00000000 :
nt!KiSystemServiceCopyEnd+0x13 (TrapFrame @
ffffd000'271d4b00)
00000000'1e22f178 00000000'00000000 : 00000000'00000000
00000000'00000000
00000000'00000000 00000000'00000000 : 0x00007ffe'f42b5c24

There is a lot of information about the thread, such as its priority, stack details, user and kernel times, and much more. You'll look at many of these details throughout this chapter and in **Chapter 5**, "**Memory management**," and **Chapter 6**, "**I/O system**."

EXPERIMENT: VIEWING THREAD INFORMATION WITH TLIST

The following output is the detailed display of a process produced by using `tlist` in the Debugging Tools for Windows. (Make sure you run `tlist` from the same "bitness" as the target process.) Notice that the thread list shows `Win32StartAddr`. This is the address passed to the `CreateThread` function by the application. All the other utilities (except Process Explorer) that show the thread start address show the actual start address (a function in Ntdll.dll), not the application-specified start address.

The following output is from running `tlist` on Word 2016 (truncated):

Click here to view code image

```
C:\Dbg\x86>tlist winword
 120 WINWORD.EXE       Chapter04.docm - Word
   CWD:     C:\Users\pavely\Documents\
   CmdLine: "C:\Program Files (x86)\Microsoft
Office\Root\Office16\WINWORD.EXE" /n
"D:\OneDrive\WindowsInternalsBook\7thEdition\Chapter04\Chapter04.docm
   VirtualSize:   778012 KB   PeakVirtualSize:   832680 KB
   WorkingSetSize:185336 KB   PeakWorkingSetSize:227144 KB
   NumberOfThreads: 45
   12132 Win32StartAddr:0x00921000 LastErr:0x00000000 State:Waiting
   15540 Win32StartAddr:0x6cc2fdd8 LastErr:0x00000000 State:Waiting
   7096 Win32StartAddr:0x6cc3c6b2 LastErr:0x00000006 State:Waiting
   17696 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   17492 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   4052 Win32StartAddr:0x70aa5cf7 LastErr:0x00000000 State:Waiting
   14096 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
   6220 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
   7204 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   1196 Win32StartAddr:0x6ea016c0 LastErr:0x00000057 State:Waiting
   8848 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
   3352 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   11612 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   17420 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   13612 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   15052 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   ...
   12080 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   9456 Win32StartAddr:0x77c1c6d0 LastErr:0x00002f94 State:Waiting
   9808 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   16208 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   9396 Win32StartAddr:0x77c1c6d0 LastErr:0x00000000 State:Waiting
   2688 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
   9100 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
   18364 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
```

```
   11180 Win32StartAddr:0x70aa41d4 LastErr:0x00000000 State:Waiting
 16.0.6741.2037 shp  0x00920000  C:\Program Files (x86)\Microsoft
Office\Root\
Office16\WINWORD.EXE
 10.0.10586.122 shp  0x77BF0000  C:\windows\SYSTEM32\ntdll.dll
   10.0.10586.0 shp  0x75540000  C:\windows\SYSTEM32\KERNEL32.DLL
 10.0.10586.162
shp  0x77850000  C:\windows\SYSTEM32\KERNELBASE.dll
  10.0.10586.63 shp  0x75AF0000  C:\windows\SYSTEM32\ADVAPI32.dll
...
   10.0.10586.0 shp  0x68540000  C:\Windows\SYSTEM32\VssTrace.DLL
   10.0.10586.0 shp  0x5C390000  C:\Windows\SYSTEM32\adsldpc.dll
 10.0.10586.122 shp  0x5DE60000  C:\Windows\SYSTEM32\taskschd.dll
   10.0.10586.0 shp  0x5E3F0000  C:\Windows\SYSTEM32\srmstormod.dll
   10.0.10586.0 shp  0x5DCA0000  C:\Windows\SYSTEM32\srmscan.dll
   10.0.10586.0 shp  0x5D2E0000  C:\Windows\SYSTEM32\msdrm.dll
   10.0.10586.0 shp  0x711E0000  C:\Windows\SYSTEM32\srm_ps.dll
   10.0.10586.0 shp  0x56680000  C:\windows\System32\OpcServices.dll
                0x5D240000  C:\Program Files (x86)\Common Files\Microsoft
Shared\Office16\WXPNSE.DLL
 16.0.6701.1023 shp  0x77E80000  C:\Program Files (x86)\Microsoft
Office\Root\Office16\GROOVEEX.DLL
   10.0.10586.0 shp  0x693F0000  C:\windows\system32\dataexchange.dll
```

The TEB, illustrated in **Figure 4-3**, is one of the data structures explained in this section that exists in the process address space (as opposed to the system space). Internally, it is made up of a header called the *Thread Information Block* (*TIB*), which mainly existed for compatibility with OS/2 and Win9*x* applications. It also allows exception and stack information to be kept into a smaller structure when creating new threads by using an initial TIB.

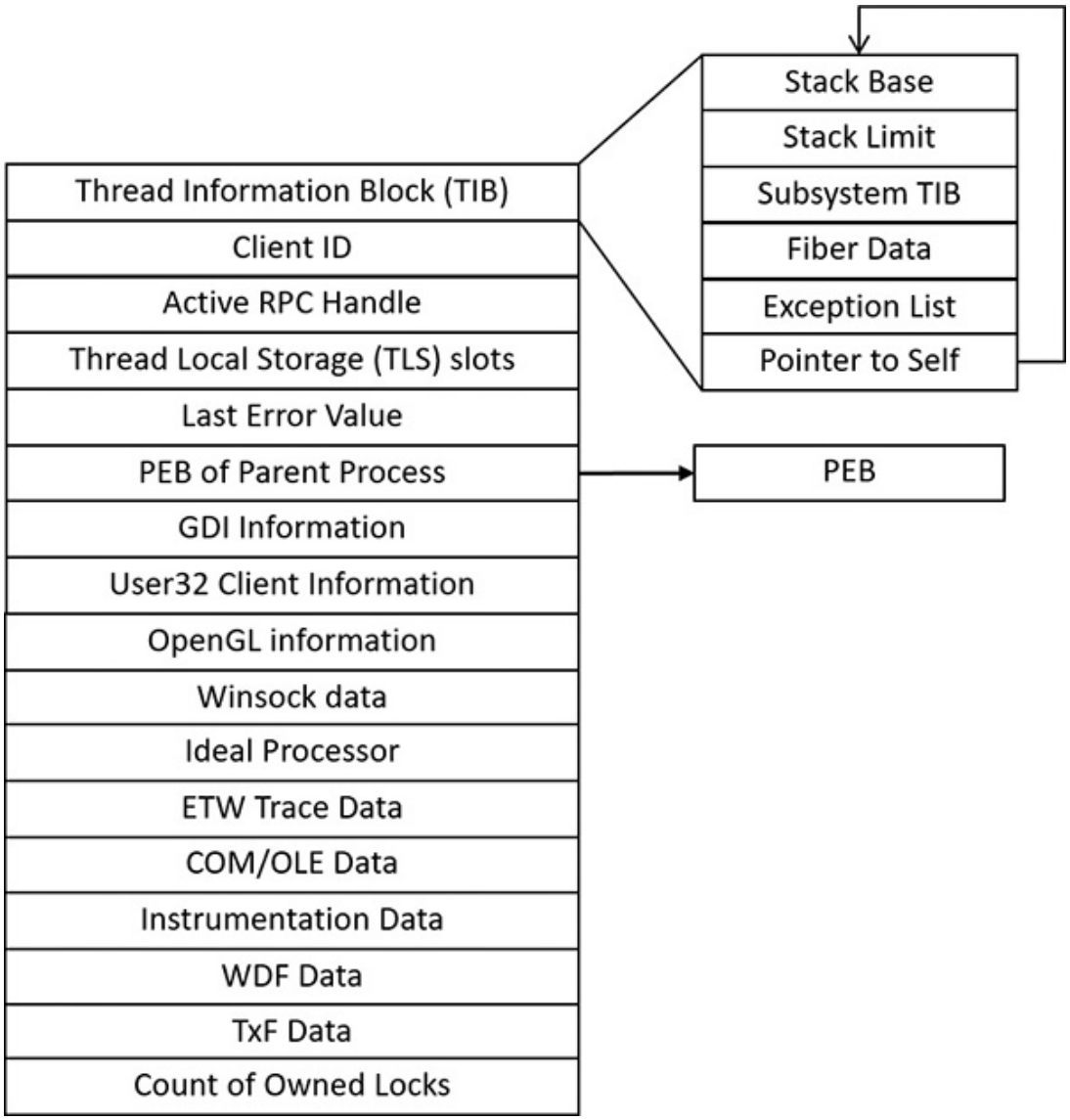


FIGURE 4-3 Important fields of the thread environment block.

The TEB stores context information for the image loader and various Windows DLLs. Because these components run in user mode, they need a data structure writable from user mode. That's why this structure exists in the process address space instead of in the system space, where it would be writable only from kernel mode. You can find the address of the TEB with the kernel debugger `!thread` command.

EXPERIMENT: EXAMINING THE TEB

You can dump the TEB structure with the `!teb` command in the kernel or user-mode debugger. The command can be used on its own to dump the TEB for the current thread of the debugger or with a TEB address to get it for an arbitrary thread. In case of a kernel debugger, the current process must be set before issuing the command on a TEB address so that the correct process context is used.

To view the TEB with a user-mode debugger, follow these steps. (You'll learn how to view the TEB using a kernel debugger in the next experiment.)

1. Open WinDbg.

2. Open the **File** menu and choose **Run Executable**.

3. Navigate to c:\windows\system32\Notepad.exe. The debugger should break at the initial breakpoint.

4. Issue the **`!teb`** command to view the TEB of the only thread existing at the moment (the example is from 64 bit Windows):

Click here to view code image

```
0:000> !teb
TEB at 000000ef125c1000
    ExceptionList:       0000000000000000
    StackBase:           000000ef12290000
    StackLimit:          000000ef1227f000
    SubSystemTib:        0000000000000000
    FiberData:           0000000000001e00
    ArbitraryUserPointer: 0000000000000000
    Self:                000000ef125c1000
    EnvironmentPointer:   0000000000000000
    ClientId:            00000000000021bc . 0000000000001b74
    RpcHandle:           0000000000000000
    Tls Storage:         00000266e572b600
```

PEB Address: 000000ef125c0000
LastErrorValue: 0
LastStatusValue: 0
Count Owned Locks: 0
HardErrorMode: 0

5. Enter the `g` command or press **F5** to proceed with running Notepad.

6. In Notepad, open the **File** menu and choose **Open**. Then click **Cancel** to dismiss the Open File dialog box.

7. Press **Ctrl+Break** or open the **Debug** menu and choose **Break** to forcibly break into the process.

8. Enter the `~` (tilde) command to show all threads in the process. You should see something like this:

Click here to view code image

```
0:005> ~
  0  Id: 21bc.1b74 Suspend: 1 Teb: 000000ef'125c1000 Unfrozen
  1  Id: 21bc.640 Suspend: 1 Teb: 000000ef'125e3000 Unfrozen
  2  Id: 21bc.1a98 Suspend: 1 Teb: 000000ef'125e5000 Unfrozen
  3  Id: 21bc.860 Suspend: 1 Teb: 000000ef'125e7000 Unfrozen
  4  Id: 21bc.28e0 Suspend: 1 Teb: 000000ef'125c9000 Unfrozen
. 5  Id: 21bc.23e0 Suspend: 1 Teb: 000000ef'12400000 Unfrozen
  6  Id: 21bc.244c Suspend: 1 Teb: 000000ef'125eb000 Unfrozen
  7  Id: 21bc.168c Suspend: 1 Teb: 000000ef'125ed000 Unfrozen
  8  Id: 21bc.1c90 Suspend: 1 Teb: 000000ef'125ef000 Unfrozen
  9  Id: 21bc.1558 Suspend: 1 Teb: 000000ef'125f1000 Unfrozen
 10  Id: 21bc.a64 Suspend: 1 Teb: 000000ef'125f3000 Unfrozen
 11  Id: 21bc.20c4 Suspend: 1 Teb: 000000ef'125f5000 Unfrozen
 12  Id: 21bc.1524 Suspend: 1 Teb: 000000ef'125f7000 Unfrozen
 13  Id: 21bc.1738 Suspend: 1 Teb: 000000ef'125f9000 Unfrozen
 14  Id: 21bc.f48 Suspend: 1 Teb: 000000ef'125fb000 Unfrozen
 15  Id: 21bc.17bc Suspend: 1 Teb: 000000ef'125fd000 Unfrozen
```

9. Each thread shows its TEB address. You can examine a specific thread by specifying its TEB address to the `!teb` command. Here's an example for thread 9 from the preceding output:

Click here to view code image

```
0:005> !teb 000000ef'125f1000
TEB at 000000ef125f1000
    ExceptionList:        0000000000000000
    StackBase:            000000ef13400000
    StackLimit:           000000ef133ef000
    SubSystemTib:         0000000000000000
    FiberData:            0000000000001e00
    ArbitraryUserPointer: 0000000000000000
    Self:                 000000ef125f1000
    EnvironmentPointer:   0000000000000000
    ClientId:             00000000000021bc . 0000000000001558
    RpcHandle:            0000000000000000
    Tls Storage:          00000266ea1af280
    PEB Address:          000000ef125c0000
    LastErrorValue:       0
    LastStatusValue:      c0000034
    Count Owned Locks:    0
    HardErrorMode:        0
```

10. Of course, it's possible to view the actual structure with the TEB address (truncated to conserve space):

Click here to view code image

```
0:005> dt ntdll!_teb 000000ef'125f1000
   +0x000 NtTib            : _NT_TIB
   +0x038 EnvironmentPointer : (null)
   +0x040 ClientId         : _CLIENT_ID
   +0x050 ActiveRpcHandle  : (null)
   +0x058 ThreadLocalStoragePointer : 0x00000266'ea1af280 Void
   +0x060 ProcessEnvironmentBlock : 0x000000ef'125c0000 _PEB
```

+0x068 LastErrorValue : 0
+0x06c CountOfOwnedCriticalSections : 0
...
+0x1808 LockCount : 0
+0x180c WowTebOffset : 0n0
+0x1810 ResourceRetValue : 0x00000266'ea2a5e50 Void
+0x1818 ReservedForWdf : (null)
+0x1820 ReservedForCrt : 0
+0x1828 EffectiveContainerId : _GUID {00000000-0000-0000-0000-000000000000}

EXPERIMENT: EXAMINING THE TEB WITH A KERNEL DEBUGGER

Follow these steps to view the TEB with a kernel debugger:

1. Find the process for which a thread's TEB is of interest. For example, the following looks for explorer.exe processes and lists its threads with basic information (truncated):

Click here to view code image

```
lkd> !process 0 2 explorer.exe
PROCESS ffffe0012bea7840
    SessionId: 2  Cid: 10d8    Peb: 00251000  ParentCid: 10bc
    DirBase: 76e12000  ObjectTable: ffffc000e1ca0c80  HandleCount:
<Data Not
Accessible>
    Image: explorer.exe

        THREAD ffffe0012bf53080  Cid 10d8.10dc  Teb: 0000000000252000
Win32Thread: ffffe0012c1532f0 WAIT: (WrUserRequest) UserMode
Non-Alertable
            ffffe0012c257fe0  SynchronizationEvent

        THREAD ffffe0012a30f080  Cid 10d8.114c  Teb: 0000000000266000
Win32Thread: ffffe0012c2e9a20 WAIT: (UserRequest) UserMode
```

Alertable
ffffe0012bab85d0 SynchronizationEvent

THREAD ffffe0012c8bd080 Cid 10d8.1178 Teb: 000000000026c000 Win32Thread: ffffe0012a801310 WAIT: (UserRequest) UserMode Alertable
ffffe0012bfd9250 NotificationEvent
ffffe0012c9512f0 NotificationEvent
ffffe0012c876b80 NotificationEvent
ffffe0012c010fe0 NotificationEvent
ffffe0012d0ba7e0 NotificationEvent
ffffe0012cf9d1e0 NotificationEvent
...

THREAD ffffe0012c8be080 Cid 10d8.1180 Teb: 0000000000270000 Win32Thread: 0000000000000000 WAIT: (UserRequest) UserMode Alertable
fffff80156946440 NotificationEvent

THREAD ffffe0012afd4040 Cid 10d8.1184 Teb: 0000000000272000 Win32Thread: ffffe0012c7c53a0 WAIT: (UserRequest) UserMode Non-Alertable
ffffe0012a3dafe0 NotificationEvent
ffffe0012c21ee70 Semaphore Limit 0xffff
ffffe0012c8db6f0 SynchronizationEvent

THREAD ffffe0012c88a080 Cid 10d8.1188 Teb: 0000000000274000 Win32Thread: 0000000000000000 WAIT: (UserRequest) UserMode Alertable
ffffe0012afd4920 NotificationEvent
ffffe0012c87b480 SynchronizationEvent
ffffe0012c87b400 SynchronizationEvent
...

2. If more than one explorer.exe process exists, select one arbitrarily for the following steps.

3. Each thread shows the address of its TEB. Because the TEB is in user space, the address has meaning only in the context of the relevant process. You need to switch to the process/thread as seen by the debugger. Select the first thread of explorer because its kernel stack is probably resident in physical memory. Otherwise, you'll get an error.

Click here to view code image

```
lkd> .thread /p ffffe0012bf53080
Implicit thread is now ffffe001'2bf53080
Implicit process is now ffffe001'2bea7840
```

4. This switches the context to the specified thread (and by extension, the process). Now you can use the `!teb` command with the TEB address listed for that thread:

Click here to view code image

```
lkd> !teb 0000000000252000
TEB at 0000000000252000
    ExceptionList:       0000000000000000
    StackBase:           00000000000d0000
    StackLimit:          00000000000c2000
    SubSystemTib:        0000000000000000
    FiberData:           0000000000001e00
    ArbitraryUserPointer: 0000000000000000
    Self:                0000000000252000
    EnvironmentPointer:  0000000000000000
    ClientId:            00000000000010d8 . 00000000000010dc
    RpcHandle:           0000000000000000
    Tls Storage:         0000000009f73f30
    PEB Address:         0000000000251000
    LastErrorValue:      0
    LastStatusValue:     c0150008
    Count Owned Locks:   0
    HardErrorMode:       0
```

The `CSR_THREAD`, illustrated in **Figure 4-4** is analogous to the data structure of `CSR_PROCESS`, but it's applied to threads. As you might recall, this is maintained by each Csrss process within a session and identifies the Windows subsystem threads running within it. `CSR_THREAD` stores a handle that Csrss keeps for the thread, various flags, the client ID (thread ID and process ID), and a copy of the thread's creation time. Note that threads are registered with Csrss when they send their first message to Csrss, typically due to some API that requires notifying Csrss of some operation or condition.

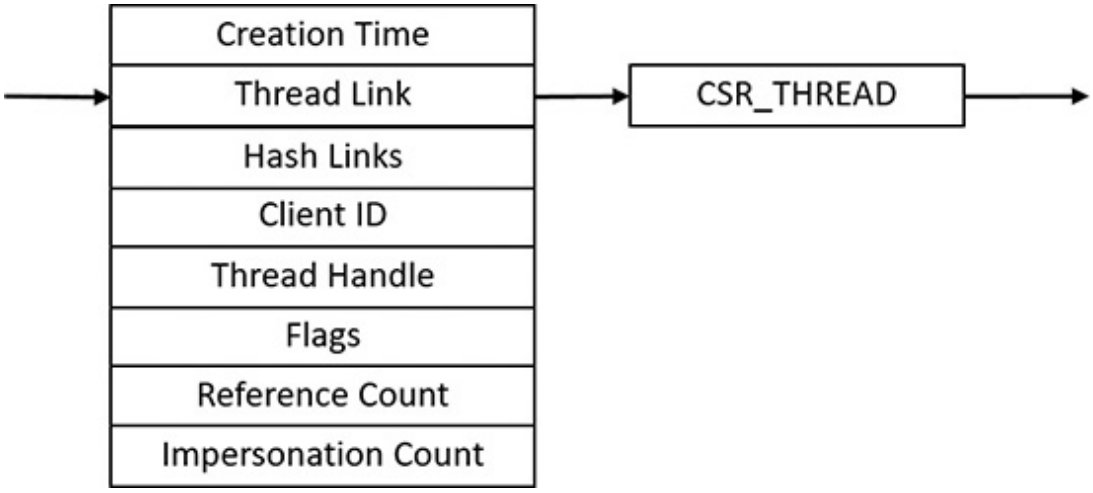


FIGURE 4-4 Fields of the CSR thread.

EXPERIMENT: EXAMINING THE CSR_THREAD

You can dump the `CSR_THREAD` structure with the `dt` command in the kernel-mode debugger while the debugger context is set to a Csrss process. Follow the instructions in the experiment "**Examining the CSR_PROCESS**" in **Chapter 3** to perform this operation. Here is an example output from Windows 10 x64 system:

Click here to view code image

```
lkd> dt csrss!_csr_thread
   +0x000 CreateTime      : _LARGE_INTEGER
   +0x008 Link            : _LIST_ENTRY
   +0x018 HashLinks       : _LIST_ENTRY
   +0x028 ClientId        : _CLIENT_ID
   +0x038 Process         : Ptr64 _CSR_PROCESS
   +0x040 ThreadHandle    : Ptr64 Void
   +0x048 Flags           : Uint4B
   +0x04c ReferenceCount  : Int4B
   +0x050 ImpersonateCount : Uint4B
```

Finally, the `W32THREAD` structure, illustrated in **Figure 4-5**, is analogous to the data structure of `W32PROCESS`, but it's applied to threads This structure mainly contains information useful for the GDI subsystem (brushes and Device Context attributes) and DirectX, as well as for the User Mode Print Driver (UMPD) framework that vendors use to write user-mode printer drivers. Finally, it contains a rendering state useful for desktop compositing and anti-aliasing.

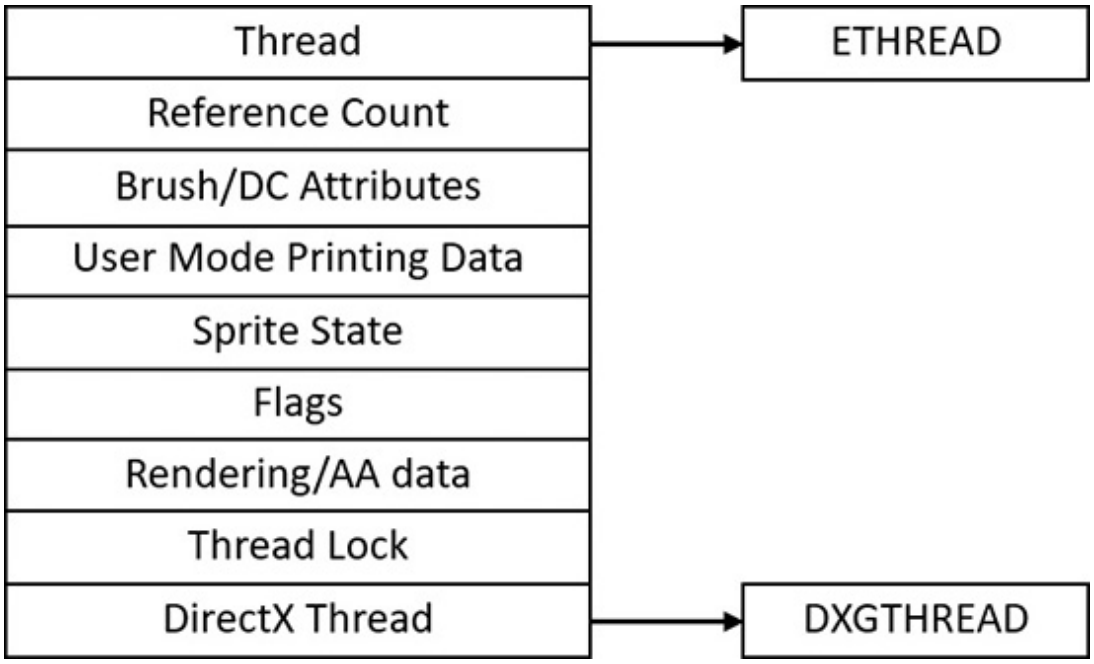


FIGURE 4-5 Fields of the Win32k thread.

Birth of a thread

A thread's life cycle starts when a process (in the context of some thread, such as the thread running the main function) creates a new thread. The request filters down to the Windows executive, where the process manager allocates space for a thread object and calls the kernel to initialize the thread control block (`KTHREAD`). As mentioned, the various thread-creation functions eventually end up at `CreateRemoteThreadEx`. The following steps are taken inside this function in Kernel32.dll to create a Windows thread:

1. The function converts the Windows API parameters to native flags and builds a native structure describing object parameters (`OBJECT_ATTRIBUTES`, described in Chapter 8 in Part 2).

2. It builds an attribute list with two entries: client ID and TEB address. (For more information on attribute lists, see the section "**Flow of CreateProcess**" in **Chapter 3**.)

3. It determines whether the thread is created in the calling process or another process indicated by the handle passed in. If the handle is equal to the pseudo handle returned from `GetCurrentProcess` (with a value of `-1`), then it's the same process. If the process handle is differ-

ent, it could still be a valid handle to the same process, so a call is made to `NtQueryInformation-Process` (in Ntdll) to find out whether that is indeed the case.

4. It calls `NtCreateThreadEx` (in Ntdll) to make the transition to the executive in kernel mode and continues inside a function with the same name and arguments.

5. `NtCreateThreadEx` (inside the executive) creates and initializes the user-mode thread context (its structure is architecture-specific) and then calls `PspCreateThread` to create a suspended executive thread object. (For a description of the steps performed by this function, see the descriptions of stage 3 and stage 5 in **Chapter 3** in the section "**Flow of CreateProcess**.") Then the function returns, eventually ending back in user mode at `CreateRemoteThreadEx`.

6. `CreateRemoteThreadEx` allocates an activation context for the thread used by side-by-side assembly support. It then queries the activation stack to see if it requires activation and activates it if needed. The activation stack pointer is saved in the new thread's TEB.

7. Unless the caller created the thread with the `CREATE_SUSPENDED` flag set, the thread is now resumed so that it can be scheduled for execution. When the thread starts running, it executes the steps described in **Chapter 3** in the section "**Stage 7: performing process initialization in the context of the new process**" before calling the actual user's specified start address.

8. The thread handle and the thread ID are returned to the caller.

Examining thread activity

Examining thread activity is especially important if you are trying to determine why a process that is hosting multiple services is running (such as Svchost.exe, Dllhost.exe, or Lsass.exe) or why a process has stopped responding.

There are several tools that expose various elements of the state of Windows threads: WinDbg (in user-process attach and kernel-debugging mode), Performance Monitor, and Process Explorer. (The tools that show thread-scheduling information are listed in the section "**Thread scheduling**.")

To view the threads in a process with Process Explorer, select a process and double-click it to open its Properties dialog box. Alternatively, right-click the process and select the **Properties** menu item. Then click the **Threads** tab. This tab shows a list of the threads in the process and four columns of information for each thread: its ID, the percentage of CPU consumed (based on the refresh interval configured), the number of cycles charged to the thread, and the thread start address. You can sort by any of these four columns.

New threads that are created are highlighted in green, and threads that exit are highlighted in red. (To configure the highlight duration, open the **Options** menu and choose **Difference Highlight Duration**.) This might be helpful to discover unnecessary thread creation occurring in a process. (In general, threads should be created at process startup, not every time a request is processed inside a process.)

As you select each thread in the list, Process Explorer displays the thread ID, start time, state, CPU time counters, number of cycles charged, number of context switches, the ideal processor and its group, and the I/O priority, memory priority, and base and current (dynamic) priority. There is a Kill button, which terminates an individual thread, but this should be used with extreme care. Another option is the Suspend button, which prevents the thread from forward execution and thus prevents a runaway thread from consuming CPU time. However, this can also lead to deadlocks and should be used with the same care as the Kill button. Finally, the Permissions button allows you to view the security descriptor of the thread. (See **Chapter 7**, "**Security**," for more information on security descriptors.)

Unlike Task Manager and all other process/processor monitoring tools, Process Explorer uses the clock cycle counter designed for thread run-

time accounting (described later in this chapter) instead of the clock interval timer, so you will see a significantly different view of CPU consumption using Process Explorer. This is because many threads run for such a short time that they are seldom (if ever) the currently running thread when the clock interval timer interrupt occurs. As a result, they are not charged for much of their CPU time, leading clock-based tools to perceive a CPU usage of 0 percent. On the other hand, the total number of clock cycles represents the actual number of processor cycles that each thread in the process accrued. It is independent of the clock interval timer's resolution because the count is maintained internally by the processor at each cycle and updated by Windows at each interrupt entry. (A final accumulation is done before a context switch.)

The thread start address is displayed in the form `module!function`, where `module` is the name of the .EXE or .DLL. The function name relies on access to symbol files for the module (see the section "**Experiment: Viewing process details with Process Explorer**" in **Chapter 1**, "**Concepts and tools**"). If you are unsure what the module is, click the **Module** button to open an Explorer file Properties dialog box for the module containing the thread's start address (for example, the .EXE or .DLL).

NOTE

For threads created by the Windows `CreateThread` function, Process Explorer displays the function passed to `CreateThread`, not the actual thread start function. This is because all Windows threads start at a common thread startup wrapper function (`RtlUserThreadStart` in Ntdll.dll). If Process Explorer showed the actual start address, most threads in processes would appear to have started at the same address, which would not be helpful in trying to understand what code the thread was executing. However, if Process Explorer can't query the user-defined startup address (such as in the case of a protected process), it will show the wrapper function, so you will see all threads starting at `RtlUserThreadStart`.

The thread start address displayed might not be enough information to pinpoint what the thread is doing and which component within the process is responsible for the CPU consumed by the thread. This is especially true if the thread start address is a generic startup function—for example, if the function name does not indicate what the thread is actually doing. In this case, examining the thread stack might answer the question. To view the stack for a thread, double-click the thread of interest (or select it and click the **Stack** button). Process Explorer displays the thread's stack (both user and kernel, if the thread was in kernel mode).

NOTE

While the user-mode debuggers (WinDbg, Ntsd, and Cdb) permit you to attach to a process and display the user stack for a thread, Process Explorer shows both the user and kernel stack in one easy click of a button. You can also examine user and kernel thread stacks using WinDbg in local kernel debugging mode, as the next two experiments demonstrate.

When looking at 32-bit processes running on 64-bit systems as a Wow64 process (see Chapter 8 in Part 2 for more information on Wow64), Process Explorer shows both the 32-bit and 64-bit stack for threads. Because at the time of the real (64 bit) system call, the thread has been switched to a 64-bit stack and context, simply looking at the thread's 64-bit stack would reveal only half the story—the 64-bit part of the thread, with Wow64's thunking code. So, when examining Wow64 processes, be sure to take into account both the 32-bit and 64-bit stacks.

EXPERIMENT: VIEWING A THREAD STACK WITH A USER-MODE DEBUGGER

Follow these steps to attach WinDbg to a process and view thread information and its stack:

1. Run notepad.exe and WinDbg.exe.

2. In WinDbg, open the **File** menu and select **Attach to Process**.

3. Find the notepad.exe instance and click **OK** to attach. The debugger should break into Notepad.

4. List the existing threads in the process with the `~` command. Each thread shows its debugger ID, the client ID (*`ProcessID`*`.`*`ThreadID`*), its suspend count (this should be `1` most of the time, as it is suspended because of the breakpoint), the TEB address, and whether it has been frozen using a debugger command.

Click here to view code image

```
0:005> ~
   0  Id: 612c.5f68 Suspend: 1 Teb: 00000022'41da2000 Unfrozen
   1  Id: 612c.5564 Suspend: 1 Teb: 00000022'41da4000 Unfrozen
   2  Id: 612c.4f88 Suspend: 1 Teb: 00000022'41da6000 Unfrozen
   3  Id: 612c.5608 Suspend: 1 Teb: 00000022'41da8000 Unfrozen
   4  Id: 612c.cf4 Suspend: 1 Teb: 00000022'41daa000 Unfrozen
.  5  Id: 612c.9f8 Suspend: 1 Teb: 00000022'41db0000 Unfrozen
```

5. Notice the dot preceding thread 5 in the output. This is the current debugger thread. Issue the `k` command to view the call stack:

Click here to view code image

```
0:005> k
 # Child-SP          RetAddr           Call Site
00 00000022'421ff7e8 00007ff8'504d9031 ntdll!DbgBreakPoint
01 00000022'421ff7f0 00007ff8'501b8102
ntdll!DbgUiRemoteBreakin+0x51
02 00000022'421ff820 00007ff8'5046c5b4
```

KERNEL32!BaseThreadInitThunk+0x22
03 00000022'421ff850 00000000'00000000
ntdll!RtlUserThreadStart+0x34

6. The debugger injected a thread into Notepad's process that issues a breakpoint instruction (`DbgBreakPoint`). To view the call stack of another thread, use the `~ n k` command, where `n` is the thread number as seen by WinDbg. (This does not change the current debugger thread.) Here's an example for thread 2:

Click here to view code image

0:005> ~2k
 # Child-SP   RetAddr   Call Site
00 00000022'41f7f9e8 00007ff8'5043b5e8
ntdll!ZwWaitForWorkViaWorkerFactory+0x14
01 00000022'41f7f9f0 00007ff8'501b8102 ntdll!TppWorkerThread+0x298
02 00000022'41f7fe00 00007ff8'5046c5b4
KERNEL32!BaseThreadInitThunk+0x22
03 00000022'41f7fe30 00000000'00000000
ntdll!RtlUserThreadStart+0x34

7. To switch the debugger to another thread, use the `~ n s` command (again, `n` is the thread number). Let's switch to thread 0 and show its stack:

Click here to view code image

0:005> ~0s
USER32!ZwUserGetMessage+0x14:
00007ff8'502e21d4 c3   ret
0:000> k
 # Child-SP   RetAddr   Call Site
00 00000022'41e7f048 00007ff8'502d3075
USER32!ZwUserGetMessage+0x14
01 00000022'41e7f050 00007ff6'88273bb3 USER32!GetMessageW+0x25
02 00000022'41e7f080 00007ff6'882890b5 notepad!WinMain+0x27b

03 00000022'41e7f180 00007ff8'341229b8
notepad!__mainCRTStartup+0x1ad
04 00000022'41e7f9f0 00007ff8'5046c5b4
KERNEL32!BaseThreadInitThunk+0x22
05 00000022'41e7fa20 00000000'00000000
ntdll!RtlUserThreadStart+0x34

8. Note that even though a thread might be in kernel mode at the time, a user-mode debugger shows its last function that's still in user mode (`ZwUserGetMessage` in the preceding output).

EXPERIMENT: VIEWING A THREAD STACK WITH A LOCAL KERNEL-MODE DEBUGGER

In this experiment, you'll use a local kernel debugger to view a thread's stack (both user mode and kernel mode). The experiment uses one of Explorer's threads, but you can try it with other processes or threads.

1. Show all the processes running the image explorer.exe. (Note that you may see more than one instance of Explorer if the Launch Folder Windows in a Separate Process option in Explorer options is selected. One process manages the desktop and taskbar, while the other manages Explorer windows.)

Click here to view code image

lkd> !process 0 0 explorer.exe
PROCESS ffffe00197398080
    SessionId: 1 Cid: 18a0   Peb: 00320000 ParentCid: 1840
    DirBase: 17c028000 ObjectTable: ffffc000bd4aa880 HandleCount:
<Data
Not Accessible>
    Image: explorer.exe

PROCESS **`ffffe00196039080`**
    SessionId: 1 Cid: 1f30   Peb: 00290000 ParentCid: 0238
    DirBase: 24cc7b000 ObjectTable: ffffc000bbbef740 HandleCount:

<Data
Not Accessible>
   Image: explorer.exe

2. Select one instance and show its thread summary:

Click here to view code image

lkd> !process **ffffe00196039080** 2
PROCESS ffffe00196039080
   SessionId: 1 Cid: 1f30   Peb: 00290000 ParentCid: 0238
   DirBase: 24cc7b000 ObjectTable: ffffc000bbbef740 HandleCount:
<Data
Not Accessible>
   Image: explorer.exe

      THREAD **ffffe0019758f080** Cid 1f30.0718 Teb: 0000000000291000
Win32Thread: ffffe001972e3220 WAIT: (UserRequest) UserMode Non-
Alertable
         ffffe00192c08150 SynchronizationEvent

      THREAD ffffe00198911080 Cid 1f30.1aac Teb: 00000000002a1000
Win32Thread: ffffe001926147e0 WAIT: (UserRequest) UserMode Non-
Alertable
         ffffe00197d6e150 SynchronizationEvent
         ffffe001987bf9e0 SynchronizationEvent

      THREAD ffffe00199553080 Cid 1f30.1ad4 Teb: 00000000002b1000
Win32Thread: ffffe0019263c740 WAIT: (UserRequest) UserMode Non-
Alertable
         ffffe0019ac6b150 NotificationEvent
         ffffe0019a7da5e0 SynchronizationEvent

      THREAD ffffe0019b6b2800 Cid 1f30.1758 Teb: 00000000002bd000
Win32Thread: 0000000000000000 WAIT: (Suspended) KernelMode Non-
Alertable
SuspendCount 1

ffffe0019b6b2ae0 NotificationEvent

...

3. Switch to the context of the first thread in the process (you can select other threads):

Click here to view code image

```
lkd> .thread /p /r ffffe0019758f080
Implicit thread is now ffffe001'9758f080
Implicit process is now ffffe001'96039080
Loading User Symbols
.....
```

4. Now look at the thread to show its details and its call stack (addresses are truncated in the shown output):

Click here to view code image

```
lkd> !thread ffffe0019758f080
THREAD ffffe0019758f080  Cid 1f30.0718  Teb: 0000000000291000
Win32Thread :
ffffe001972e3220 WAIT : (UserRequest)UserMode Non - Alertable
ffffe00192c08150  SynchronizationEvent
Not impersonating
DeviceMap                 ffffc000b77f1f30
Owning Process            ffffe00196039080       Image : explorer.exe
Attached Process          N / A            Image : N / A
Wait Start TickCount      17415276       Ticks : 146 (0:00 : 00 : 02.281)
Context Switch Count      2788           IdealProcessor : 4
UserTime                  00 : 00 : 00.031
KernelTime                00 : 00 : 00.000
*** WARNING : Unable to verify checksum for C :
\windows\explorer.exe
Win32 Start Address
explorer!wWinMainCRTStartup(0x00007ff7b80de4a0)
Stack Init ffffd0002727cc90 Current ffffd0002727bf80
```

Base ffffd0002727d000 Limit ffffd00027277000 Call 0000000000000000 Priority 8 BasePriority 8 PriorityDecrement 0 IoPriority 2 PagePriority 5

... Call Site
... nt!KiSwapContext + 0x76
... nt!KiSwapThread + 0x15a
... nt!KiCommitThreadWait + 0x149
... nt!KeWaitForSingleObject + 0x375
... nt!ObWaitForMultipleObjects + 0x2bd
... nt!NtWaitForMultipleObjects + 0xf6
... nt!KiSystemServiceCopyEnd + 0x13 (TrapFrame @ ffffd000'2727cb00)
... ntdll!ZwWaitForMultipleObjects + 0x14
... KERNELBASE!WaitForMultipleObjectsEx + 0xef
... USER32!RealMsgWaitForMultipleObjectsEx + 0xdb
... USER32!MsgWaitForMultipleObjectsEx + 0x152
... explorerframe!SHProcessMessagesUntilEventsEx + 0x8a
... explorerframe!SHProcessMessagesUntilEventEx + 0x22
... explorerframe!CExplorerHostCreator::RunHost + 0x6d
... explorer!wWinMain + 0xa04fd
... explorer!__wmainCRTStartup + 0x1d6

Limitations on protected process threads

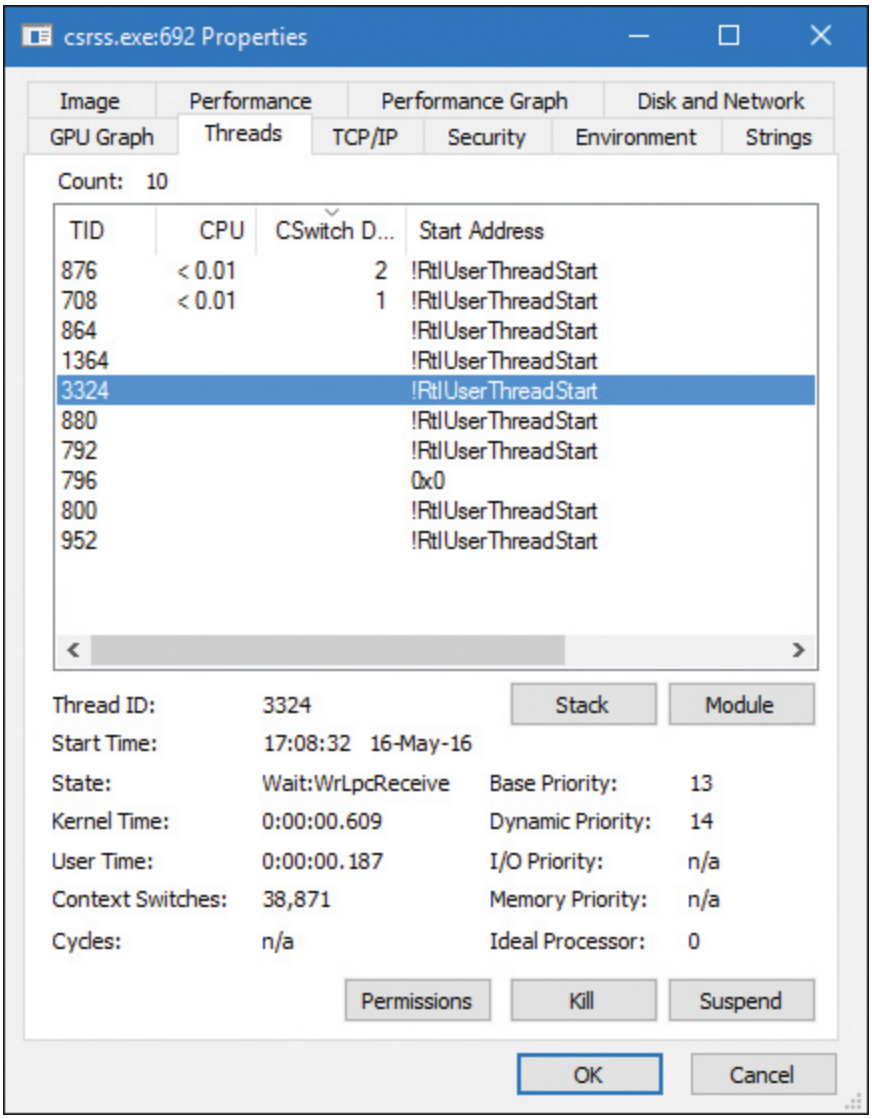
As discussed in **Chapter 3**, protected processes (classic protected or PPL) have several limitations in terms of which access rights will be granted, even to the users with the highest privileges on the system. These limitations also apply to threads inside such a process. This ensures that the actual code running inside the protected process cannot be hijacked or otherwise affected through standard Windows functions, which require access rights that are not granted for protected process threads. In fact, the only permissions granted are `THREAD_SUSPEND_RESUME` and `THREAD_SET/QUERY_LIMITED_INFORMATION`.

EXPERIMENT: VIEWING PROTECTED PROCESS THREAD INFORMATION WITH PROCESS EXPLORER

In this experiment, you'll view protected process thread information. Follow these steps:

1. Find any protected or PPL process, such as the Audiodg.exe or Csrss.exe process inside the process list.

2. Open the process's Properties dialog box and click the **Threads** tab.



3. Process Explorer doesn't show the Win32 thread start address. Instead, it displays the standard thread start wrapper inside Ntdll.dll. If you click the **Stack** button, you'll get an error, because Process Explorer needs to read the virtual memory inside the protected process, which it can't do.

4. Note that although the base and dynamic priorities are shown, the I/O and memory priorities are not (nor is Cycles), which is another ex-

ample of the limited access right `THREAD_QUERY_LIMITED_INFORMATION` versus full query information access right (`THREAD_QUERY_INFORMATION`).

5. Try to kill a thread inside a protected process. When you do, notice yet another access-denied error: recall the lack of `THREAD_TERMINATE` access.

Thread scheduling

This section describes the Windows scheduling policies and algorithms. The first subsection provides a condensed description of how scheduling works on Windows and a definition of key terms. Then Windows priority levels are described from both the Windows API and the Windows kernel points of view. After a review of the relevant Windows utilities and tools that relate to scheduling, the detailed data structures and algorithms that make up the Windows scheduling system are presented, including a description of common scheduling scenarios and how thread selection, as well as processor selection, occurs.

Overview of Windows scheduling

Windows implements a priority-driven, preemptive scheduling system. At least one of the highest-priority runnable (ready) threads always runs, with the caveat that certain high-priority threads ready to run might be limited by the processors on which they might be allowed or preferred to run on—phenomenon called *processor affinity*. Processor affinity is defined based on a given processor group, which collects up to 64 processors. By default, threads can run only on available processors within the processor group associated with the process. (This is to maintain compatibility with older versions of Windows, which supported only 64 processors). Developers can alter processor affinity by using the appropriate APIs or by setting an affinity mask in the image header, and users can use tools to change affinity at run time or at process creation. However, although multiple threads in a process can be associated with different groups, a thread on its own can run only on the processors available within its assigned group. Additionally, developers can choose to create group-aware applications, which use extended scheduling APIs to associate logical processors on different groups with the affinity of their threads. Doing so converts the process into a multigroup process that can theoretically run its threads on any available processor within the machine.

After a thread is selected to run, it runs for an amount of time called a *quantum*. A quantum is the length of time a thread is allowed to run before another thread at the same priority level is given a turn to run. Quantum values can vary from system to system and process to process for any of three reasons:

- System configuration settings (long or short quantums, variable or fixed quantums, and priority separation)
- Foreground or background status of the process
- Use of the job object to alter the quantum

These details are explained in the “Quantum” section later in this chapter.

A thread might not get to complete its quantum, however, because Windows implements a preemptive scheduler. That is, if another thread with a higher priority becomes ready to run, the currently running thread might be preempted before finishing its time slice. In fact, a thread can be selected to run next and be preempted before even beginning its quantum!

The Windows scheduling code is implemented in the kernel. There’s no single “scheduler” module or routine, however. The code is spread throughout the kernel in which scheduling-related events occur. The routines that perform these duties are collectively called the kernel’s *dispatcher*. The following events might require thread dispatching:

- A thread becomes ready to execute—for example, a thread has been newly created or has just been released from the wait state.

- A thread leaves the running state because its time quantum ends, it terminates, it yields execution, or it enters a wait state.

- A thread’s priority changes, either because of a system service call or because Windows itself changes the priority value.

- A thread’s processor affinity changes so that it will no longer run on the processor on which it was running.

At each of these junctions, Windows must determine which thread should run next on the logical processor that was running the thread, if applicable, or on which logical processor the thread should now run. After a logical processor has selected a new thread to run, it eventually performs a context switch to it. A *context switch* is the procedure of saving the volatile processor state associated with a running thread, loading another thread’s volatile state, and starting the new thread’s execution.

As noted, Windows schedules at the thread granularity level. This approach makes sense when you consider that processes don't run; rather, they only provide resources and a context in which their threads run. Because scheduling decisions are made strictly on a thread basis, no consideration is given to what process the thread belongs to. For example, if process A has 10 runnable threads, process B has 2 runnable threads, and all 12 threads are at the same priority, each thread would theoretically receive one-twelfth of the CPU time. That is, Windows wouldn't give 50 percent of the CPU to process A and 50 percent to process B.

Priority levels

To understand the thread-scheduling algorithms, one must first understand the priority levels that Windows uses. As illustrated in **Figure 4-6**, Windows uses 32 priority levels internally, ranging from 0 to 31 (31 is the highest). These values divide up as follows:

- Sixteen real-time levels (16 through 31)

- Sixteen variable levels (0 through 15), out of which level 0 is reserved for the zero page thread (described in **Chapter 5**).

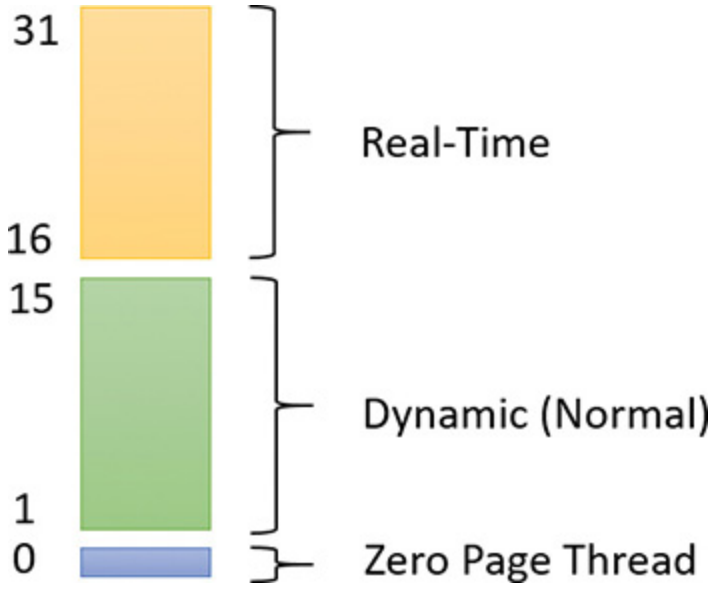


FIGURE 4-6 Thread priority levels.

Thread priority levels are assigned from two different perspectives: those of the Windows API and those of the Windows kernel. The

Windows API first organizes processes by the priority class to which they are assigned at creation (the numbers in parentheses represent the internal `PROCESS_PRIORITY_CLASS` index recognized by the kernel):

- Real-Time (4)
- High (3)
- Above Normal (6)
- Normal (2)
- Below Normal (5)
- Idle (1)

The Windows API `SetPriorityClass` allows changing a process's priority class to one of these levels.

It then assigns a relative priority of the individual threads within those processes. Here, the numbers represent a priority delta that is applied to the process base priority:

- Time-Critical (15)
- Highest (2)
- Above-Normal (1)
- Normal (0)
- Below-Normal (–1)
- Lowest (–2)
- Idle (–15)

Time-Critical and Idle levels (+15 and –15) are called *saturation values* and represent specific levels that are applied rather than true offsets. These values can be passed to the `SetThreadPriority` Windows API to change a thread's relative priority.

Therefore, in the Windows API, each thread has a base priority that is a function of its process priority class and its relative thread priority. In the kernel, the process priority class is converted to a base priority by using the `PspPriorityTable` global array and the `PROCESS_PRIORITY_CLASS` indices shown earlier, which sets priorities of 4, 8, 13, 24, 6, and 10, respectively. (This is a fixed mapping that cannot be changed.) The relative thread priority is then applied as a differential to this base priority. For example, a Highest thread will receive a thread base priority of two levels higher than the base priority of its process.

This mapping from Windows priority to internal Windows numeric priority is shown graphically in **Figure 4-7** and textually in **Table 4-1**.

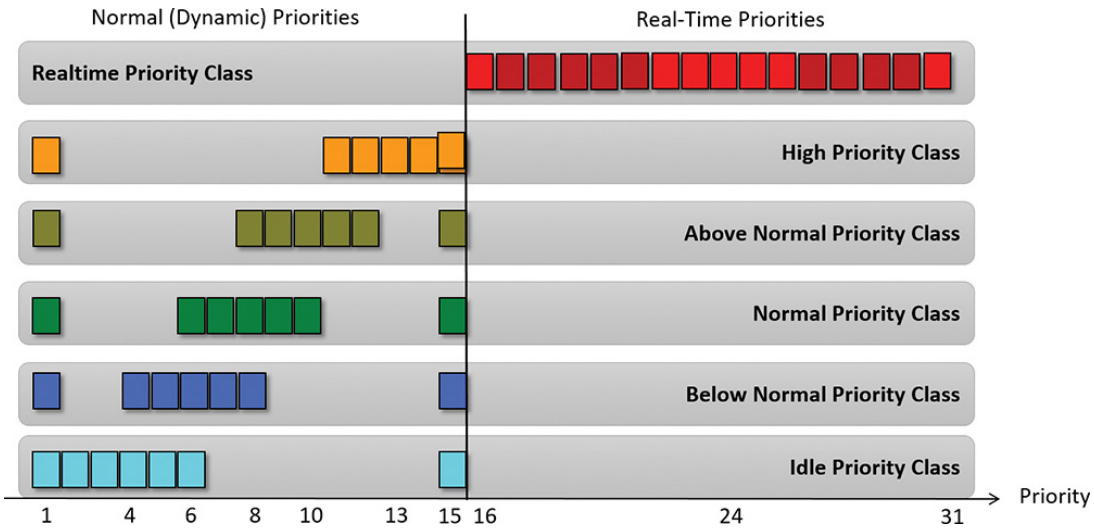


FIGURE 4-7 A graphic view of available thread priorities from a Windows API perspective.

Priority Class Relative Priority	Real-Time	High	Above-Normal	Normal	Below-Normal	Idle
Time Critical (+Saturation)	31	15	15	15	15	15
Highest (+2)	26	15	12	10	8	6
Above Normal (+1)	25	14	11	9	7	5
Normal (0)	24	13	10	8	6	4
Below Normal (-1)	23	12	9	7	5	3
Lowest (-2)	22	11	8	6	4	2
Idle (–Saturation)	16	1	1	1	1	1

TABLE 4-1 Mapping Windows kernel priorities to the Windows API

You'll note that the Time-Critical and Idle relative thread priorities maintain their respective values regardless of the process priority class (unless it is Real-Time). This is because the Windows API requests saturation of the priority from the kernel, by passing in +16 or –16 as the requested relative priority. The formula used to get these values is as follows (`HIGH_PRIORITY` equals 31):

Click here to view code image

```
If Time-Critical: ((HIGH_PRIORITY+1) / 2

If Idle: -((HIGH_PRIORITY+1) / 2
```

These values are then recognized by the kernel as a request for saturation, and the `Saturation` field in `KTHREAD` is set. For positive saturation, this causes the thread to receive the highest possible priority within its priority class (dynamic or real-time); for negative saturation, it's the lowest possible one. Additionally, future requests to change the base priority of the process will no longer affect the base priority of these threads because saturated threads are skipped in the processing code.

As shown in **Table 4-1**, threads have seven levels of possible priorities to set as viewed from the Windows API (six levels for the High priority class). The Real-Time priority class actually allows setting all priority levels between 16 and 31 (as shown in **Figure 4-7**). The values not covered by the standard constants shown in the table can be specified with the values `-7`, `-6`, `-5`, `-4`, `-3`, `3`, `4`, `5`, and `6` as an argument to

`SetThreadPriority`. (See the upcoming section "**Real-Time priorities**" for more information.)

Regardless of how the thread's priority came to be by using the Windows API (a combination of process priority class and a relative thread priority), from the point of view of the scheduler, only the final result matters. For example, priority level 10 can be obtained in two ways: a Normal priority class process (8) with a thread relative priority of Highest (+2), or an Above-Normal priority class process (10) and a Normal thread relative priority (0). From the scheduler's perspectives, these settings lead to the same value (10), so these threads are identical in terms of their priority.

Whereas a process has only a single base priority value, each thread has two priority values: current (dynamic) and base. Scheduling decisions are made based on the current priority. As explained in the upcoming section called "**Priority boosts**," under certain circumstances, the system increases the priority of threads in the dynamic range (1 through 15) for brief periods. Windows never adjusts the priority of threads in the Real-Time range (16 through 31), so they always have the same base and current priority.

A thread's initial base priority is inherited from the process base priority. A process, by default, inherits its base priority from the process that created it. You can override this behavior on the `Create-Process` function or by using the command-line `start` command. You can also change a process priority after it is created by using the `SetPriorityClass` function or by using various tools that expose that function, such as Task Manager or Process Explorer. (Right-click on the process and choose a new priority class.) For example, you can lower the priority of a CPU-intensive process so that it does not interfere with normal system activities. Changing the priority of a process changes the thread priorities up or down, but their relative settings remain the same.

Normally, user applications and services start with a normal base priority, so their initial thread typically executes at priority level 8. However,

some Windows system processes (such as the Session manager, Service Control Manager, and local security authentication process) have a base process priority slightly higher than the default for the Normal class (8). This higher default value ensures that the threads in these processes will all start at a higher priority than the default value of 8.

Real-Time priorities

You can raise or lower thread priorities within the dynamic range in any application. However, you must have the increase scheduling priority privilege (`SeIncreaseBasePriorityPrivilege`) to enter the Real-Time range. Be aware that many important Windows kernel-mode system threads run in the Real-Time priority range, so if threads spend excessive time running in this range, they might block critical system functions (such as in the memory manager, cache manager, or some device drivers).

Using the standard Windows APIs, once a process has entered the Real-Time range, all its threads (even Idle ones) must run at one of the Real-Time priority levels. It is thus impossible to mix real-time and dynamic threads within the same process through standard interfaces. This is because the `SetThreadPriority` API calls the native `NtSetInformationThread` API with the `ThreadBasePriority` information class, which allows priorities to remain only in the same range. Furthermore, this information class allows priority changes only in the recognized Windows API deltas of –2 to 2 (or Time-Critical/Idle) unless the request comes from CSRSS or another real-time process. In other words, this means that a real-time process can pick thread priorities anywhere between 16 and 31, even though the standard Windows API relative thread priorities would seem to limit its choices based on the table that was shown earlier.

As mentioned, calling `SetThreadPriority` with one of a set of special values causes a call to `NtSetInformationThread` with the `ThreadActualBasePriority` information class, the kernel base priority for the thread can be directly set, including in the dynamic range for a real-time process.

NOTE

The name *real-time* does not imply that Windows is a real-time OS in the common definition of the term. This is because Windows doesn't provide true, real-time OS facilities, such as guaranteed interrupt latency or a way for threads to obtain a guaranteed execution time. The term *real-time* really just means "higher than all the others."

Using tools to interact with priority

You can change (and view) the base-process priority with Task Manager and Process Explorer. You can kill individual threads in a process with Process Explorer (which should be done, of course, with extreme care).

You can view individual thread priorities with Performance Monitor, Process Explorer, or WinDbg. Although it might be useful to increase or decrease the priority of a process, it typically does not make sense to adjust individual thread priorities within a process because only a person who thoroughly understands the program (in other words, the developer) would understand the relative importance of the threads within the process.

The only way to specify a starting priority class for a process is with the `start` command in the Windows command prompt. If you want to have a program start every time with a specific priority, you can define a shortcut to use the `start` command by beginning the command with `cmd /c`. This runs the command prompt, executes the command on the command line, and terminates the command prompt. For example, to run Notepad in the Idle-process priority, the command is `cmd /c start /low Notepad.exe`.

EXPERIMENT: EXAMINING AND SPECIFYING PROCESS AND THREAD PRIORITIES

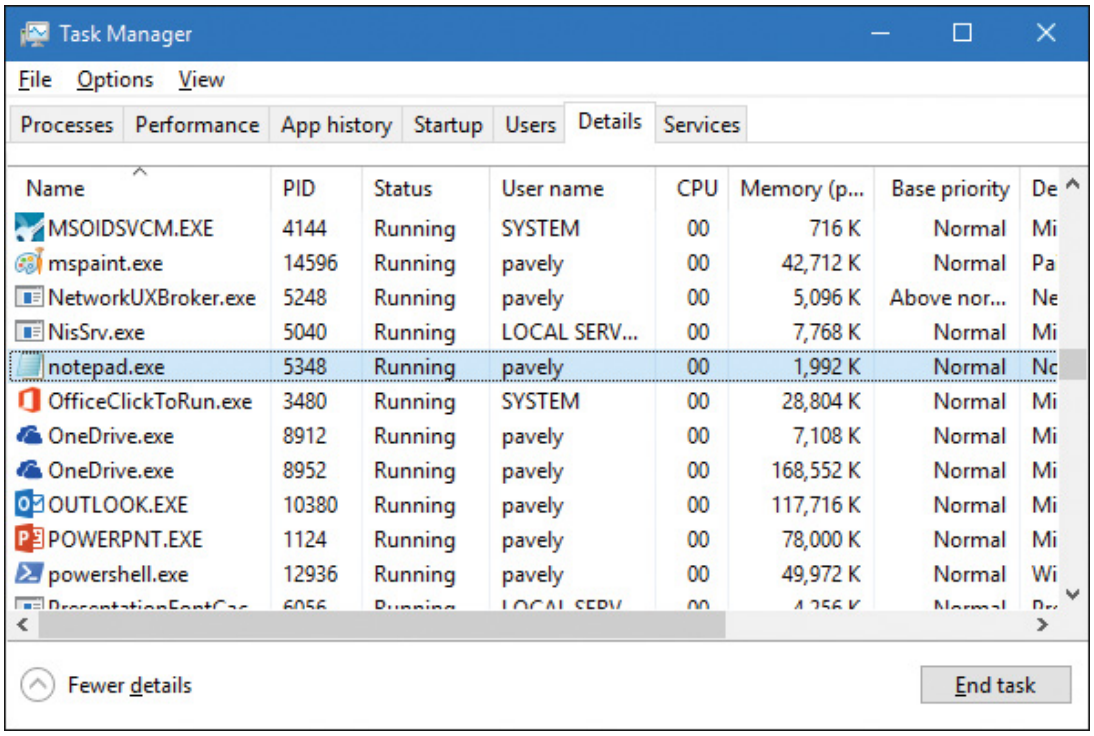
To examine and specify process and thread priorities, follow these steps:

1. Run notepad.exe normally—for example, by typing `Notepad` in a command window.

2. Open Task Manager and click to the **Details** tab.

3. Add a column named **Base Priority**. This is the name Task Manager uses for priority class.

4. Find Notepad in the list. You should see something like the following:

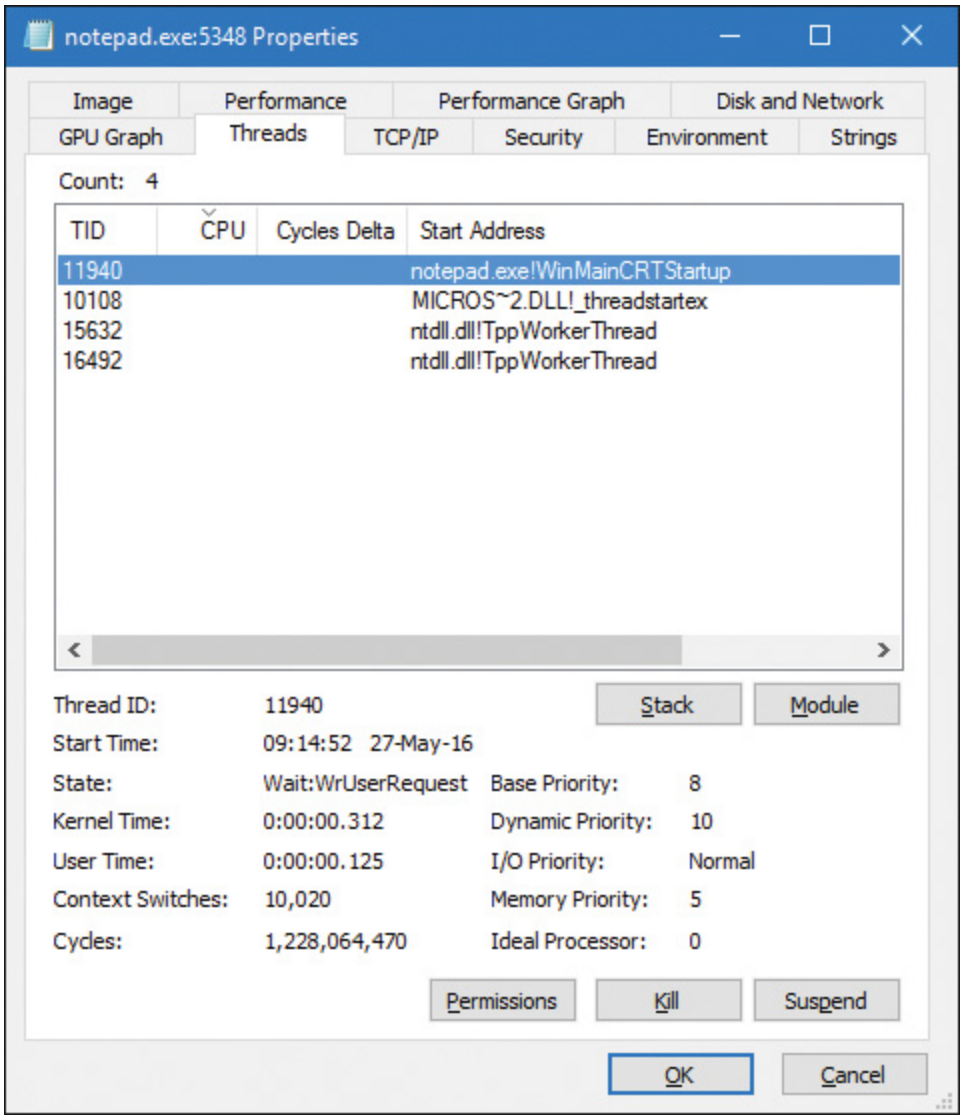


5. Notice the Notepad process running with the Normal priority class (8) and that Task Manager shows the Idle priority class as Low.

6. Open Process Explorer.

7. Double-click the **Notepad** process to show its Properties dialog box and click the **Threads** tab.

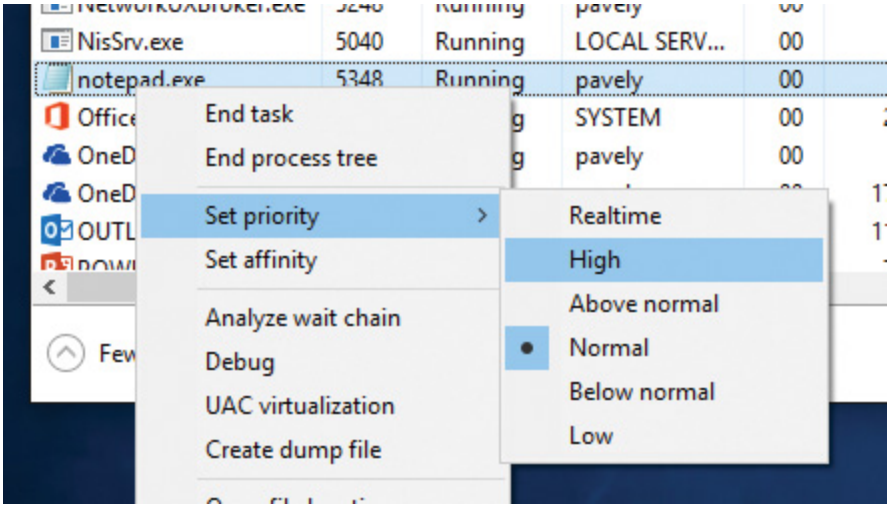
8. Select the first thread (if there's more than one). You should see something like this:



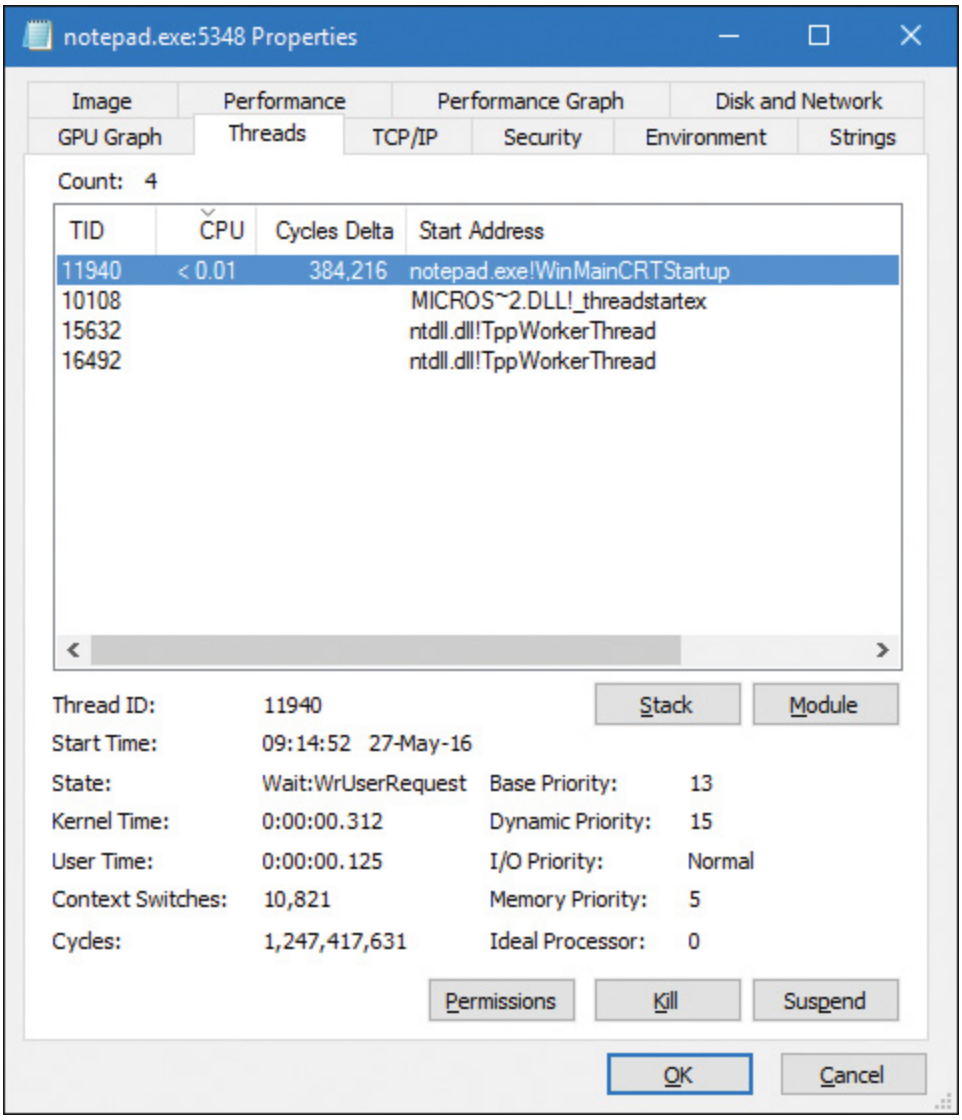
9. Notice the thread's priorities. Its base priority is 8 but its current (dynamic) priority is 10. (The reason for this priority boost is discussed in the upcoming "**Priority boosts**" section).

10. If you want, you can suspend and kill the thread. (Both operations must be used with caution, of course.)

11. In Task Manager, right-click the **Notepad** process, select **Set Priority**, and set the value to **High**, as shown here:

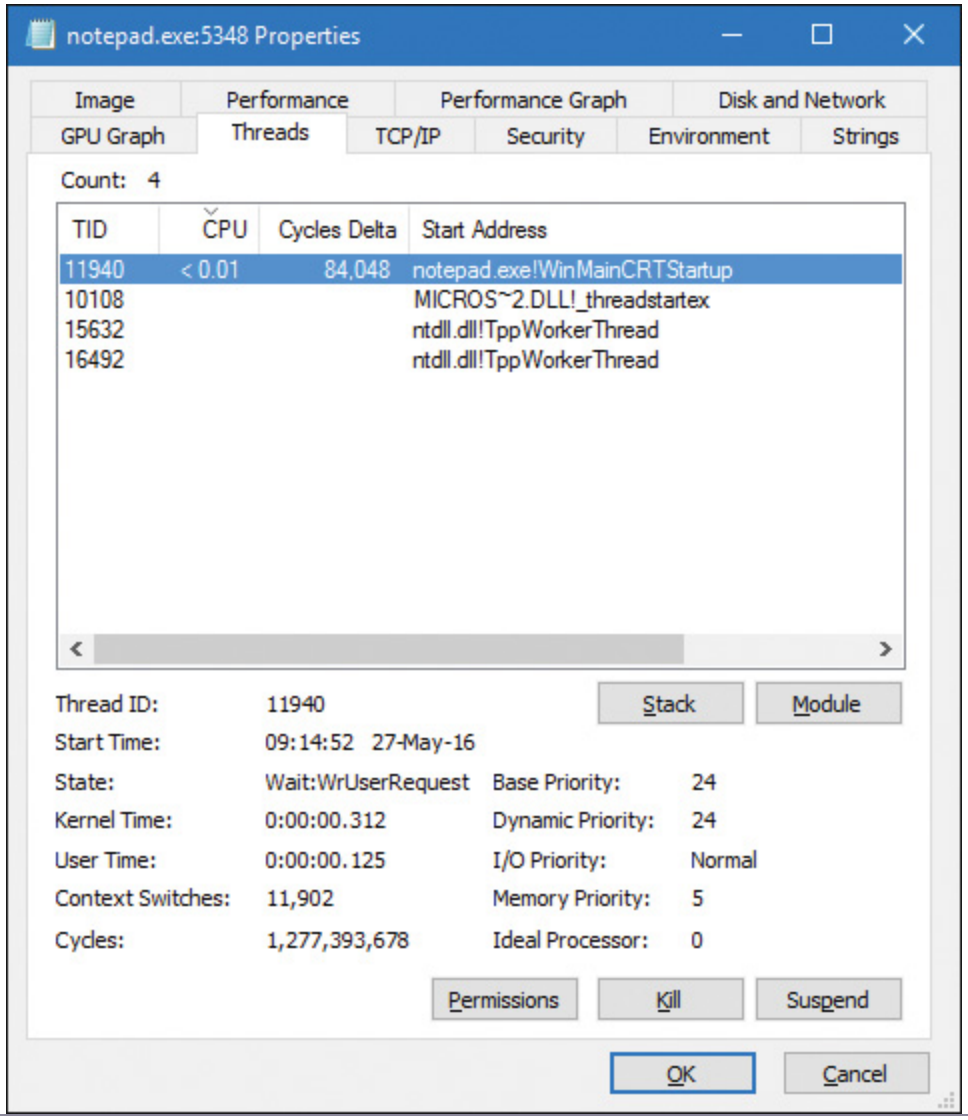


12. Accept the confirmation dialog box change and go back to Process Explorer. Notice that the thread's priority has jumped to the new base for High (13). The dynamic priority has made the same relative jump:



13. In Task Manager, change the priority class to **Realtime**. (You must be an administrator on the machine for this to succeed. Note that you can also make this change in Process Explorer.)

14. In Process Manager, notice that the base and dynamic priorities of the thread are now 24. Recall that the kernel never applies priority boosts for threads in the Real-Time priority range.



WINDOWS SYSTEM RESOURCE MANAGER

Windows Server 2012 R2 Standard Edition and higher SKUs include an optionally installable component called *Windows System Resource Manager* (*WSRM*). It permits the administrator to configure policies that specify CPU utilization, affinity settings, and memory limits (both physical and virtual) for processes. In addition, WSRM can generate resource-utilization reports that can be used for accounting and verification of service-level agreements with users.

Policies can be applied for specific applications (by matching the name of the image with or without specific command-line arguments), users,

or groups. The policies can be scheduled to take effect at certain periods or can be enabled all the time.

After you set a resource-allocation policy to manage specific processes, the WSRM service monitors CPU consumption of managed processes and adjusts process base priorities when those processes do not meet their target CPU allocations.

The physical memory limitation uses the function `SetProcessWorkingSetSizeEx` to set a hard-working set maximum. The virtual memory limit is implemented by the service checking the private virtual memory consumed by the processes. (See **Chapter 5** for an explanation of these memory limits.) If this limit is exceeded, WSRM can be configured to either kill the processes or write an entry to the event log. This behavior can be used to detect a process with a memory leak before it consumes all the available committed memory on the system. Note that WSRM memory limits do not apply to Address Windowing Extensions (AWE) memory, large page memory, or kernel memory (non-paged or paged pool). (See **Chapter 5** for more information on these terms.)

Thread states

Before looking at the thread-scheduling algorithms, you must understand the various execution states that a thread can be in. The thread states are as follows:

- **Ready** A thread in the ready state is waiting to execute or to be inswapped after completing a wait. When looking for a thread to execute, the dispatcher considers only the threads in the ready state.

- **Deferred ready** This state is used for threads that have been selected to run on a specific processor but have not actually started running there. This state exists so that the kernel can minimize the amount of time the per-processor lock on the scheduling database is held.

■ **Standby** A thread in this state has been selected to run next on a particular processor. When the correct conditions exist, the dispatcher performs a context switch to this thread. Only one thread can be in the standby state for each processor on the system. Note that a thread can be preempted out of the standby state before it ever executes (if, for example, a higher-priority thread becomes runnable before the standby thread begins execution).

■ **Running** After the dispatcher performs a context switch to a thread, the thread enters the running state and executes. The thread's execution continues until its quantum ends (and another thread at the same priority is ready to run), it is preempted by a higher-priority thread, it terminates, it yields execution, or it voluntarily enters the waiting state.

■ **Waiting** A thread can enter the waiting state in several ways: A thread can voluntarily wait for an object to synchronize its execution, the OS can wait on the thread's behalf (such as to resolve a paging I/O), or an environment subsystem can direct the thread to suspend itself. When the thread's wait ends, depending on its priority, the thread either begins running immediately or is moved back to the ready state.

■ **Transition** A thread enters the transition state if it is ready for execution but its kernel stack is paged out of memory. After its kernel stack is brought back into memory, the thread enters the ready state. (Thread stacks are discussed in **Chapter 5**.)

■ **Terminated** When a thread finishes executing, it enters this state. After the thread is terminated, the executive thread object (the data structure in system memory that describes the thread) might or might not be deallocated. The object manager sets the policy regarding when to delete the object. For example, the object remains if there are any open handles to the thread. A thread can also enter the terminated state from other states if it's killed explicitly by some other thread—for example, by calling the `TerminateThread` Windows API.

■ **Initialized** This state is used internally while a thread is being created.

Figure 4-8 shows the main state transitions for threads. The numeric values shown represent the internal values of each state and can be viewed with a tool such as Performance Monitor. The ready and deferred ready states are represented as one. This reflects the fact that the deferred ready state acts as a temporary placeholder for the scheduling routines. This is true for the standby state as well. These states are almost always very short-lived. Threads in these states always transition quickly to ready, running, or waiting.

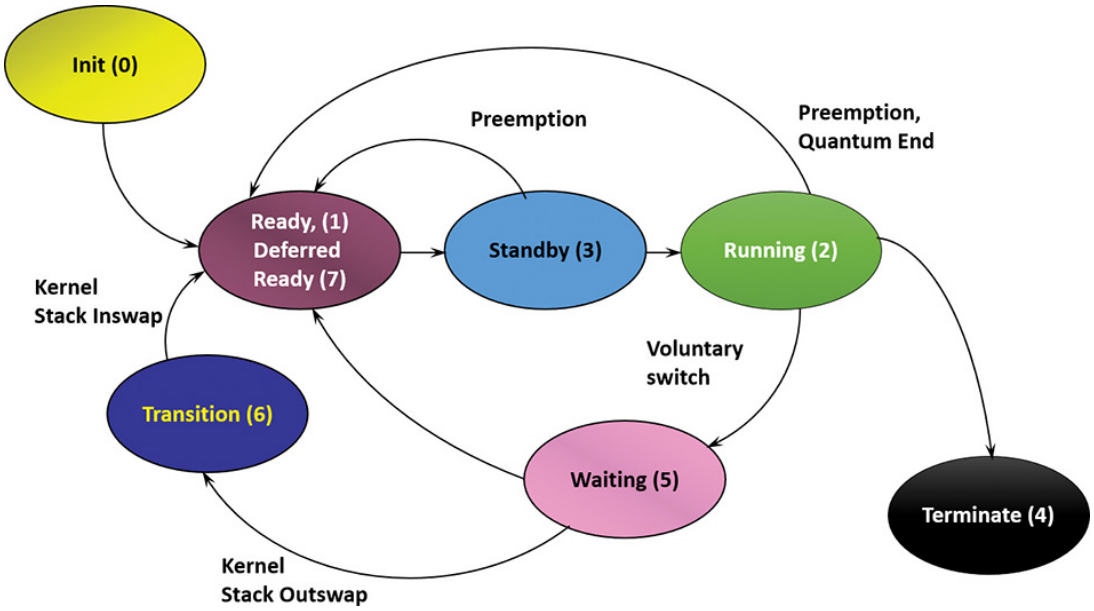


FIGURE 4-8 Thread states and transitions.

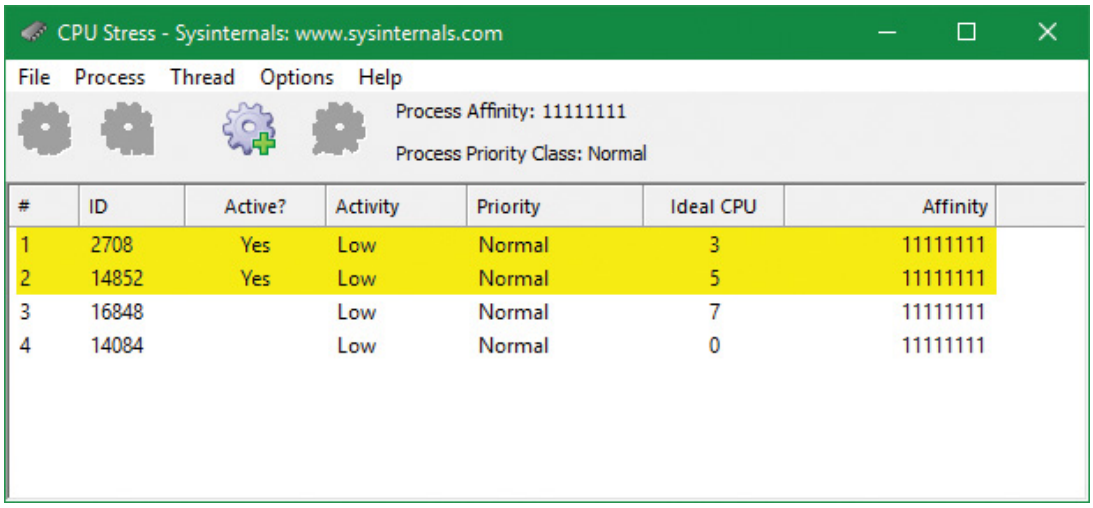
EXPERIMENT: THREAD-SCHEDULING STATE CHANGES

You can watch thread-scheduling state changes with the Performance Monitor tool in Windows. This utility can be useful when you're debugging a multithreaded application and you're unsure about the state of the threads running in the process. To watch thread-scheduling state changes by using the Performance Monitor tool, follow these steps:

1. Download the CPU Stress tool from the book's downloadable resources.

2. Run CPUSTRES.exe. Thread 1 should be active.

3. Activate thread 2 by selecting it in the list and clicking the **Activate** button or by right-clicking it and selecting **Activate** from the context menu. The tool should look something like this:



4. Click the **Start** button and type **perfmon** to start the Performance Monitor tool.

5. If necessary, select the chart view. Then remove the existing CPU counter.

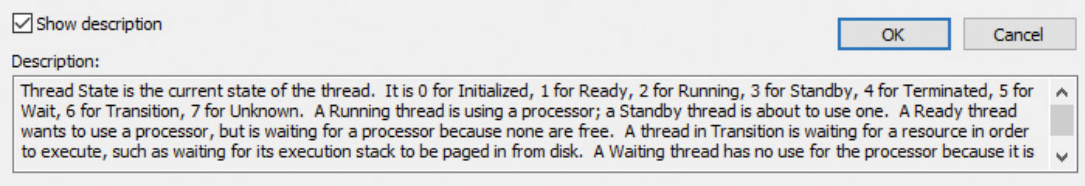
6. Right-click the graph and choose **Properties**.

7. Click the **Graph** tab and change the chart vertical scale maximum to 7. (As you saw in Figure 4-8, the various states are associated with numbers 0 through 7.) Then click **OK**.

8. Click the **Add** button on the toolbar to open the Add Counters dialog box.

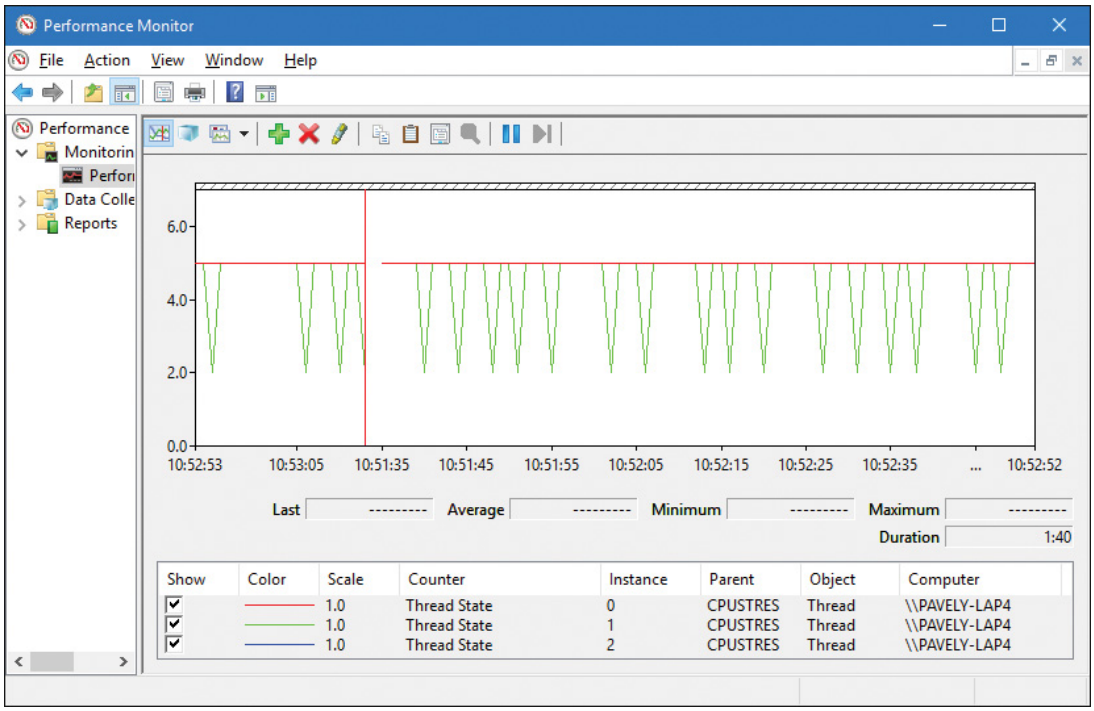
9. Select the **Thread** performance object and then select the **Thread State** counter.

10. Select the **Show Description** check box to see the definition of the values:



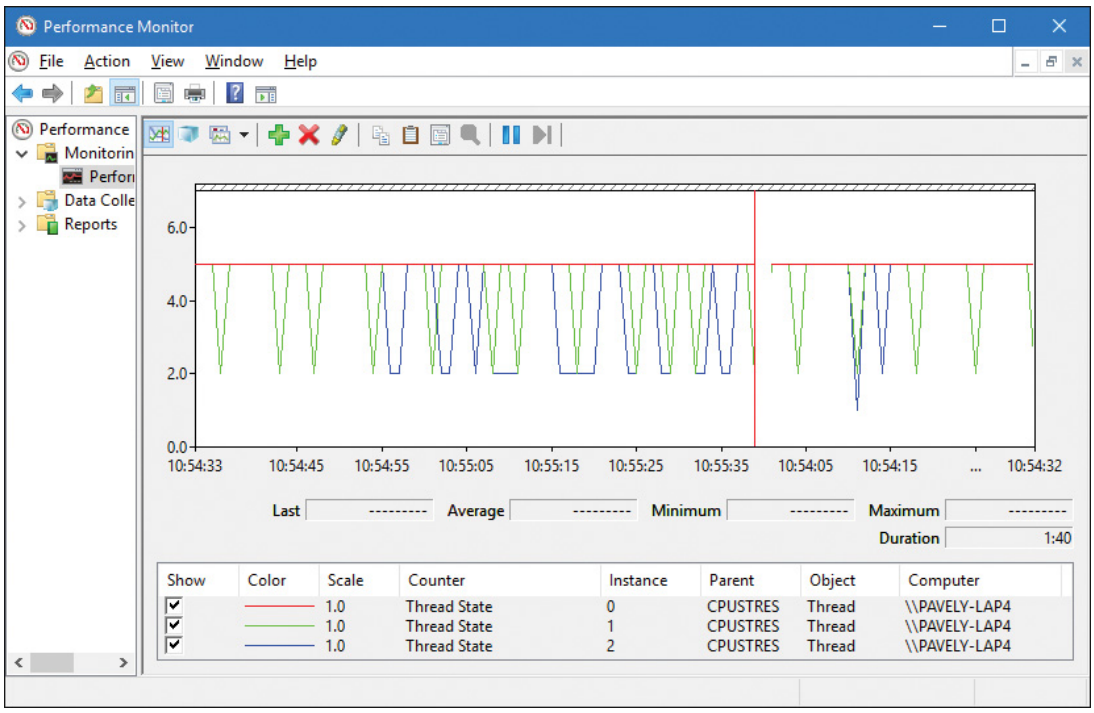
11. In the Instances box, select <**All instances**>. Then type **cpustres** and click **Search**.

12. Select the first three threads of cpustres (**cpustres/0**, **cpustres/1**, and **cpustres/2**) and click the **Add** >> button. Then click **OK**. Thread 0 should be in state 5 (waiting), because that's the GUI thread and it's waiting for user input. Threads 1 and 2 should be alternating between states 2 and 5 (running and waiting). (Thread 1 may be hiding thread 2 as they're running with the same activity level and the same priority.)

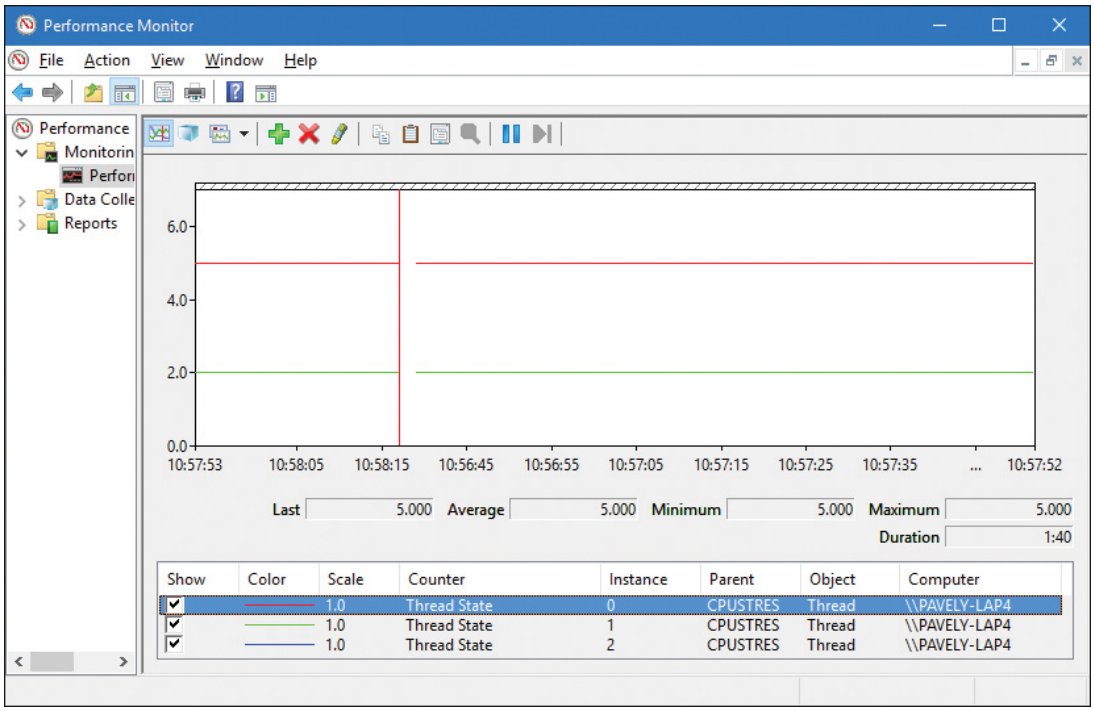


13. Go back to CPU Stress, right-click thread 2, and choose **Busy** from the activity context menu. You should see thread 2 in state 2 (running)

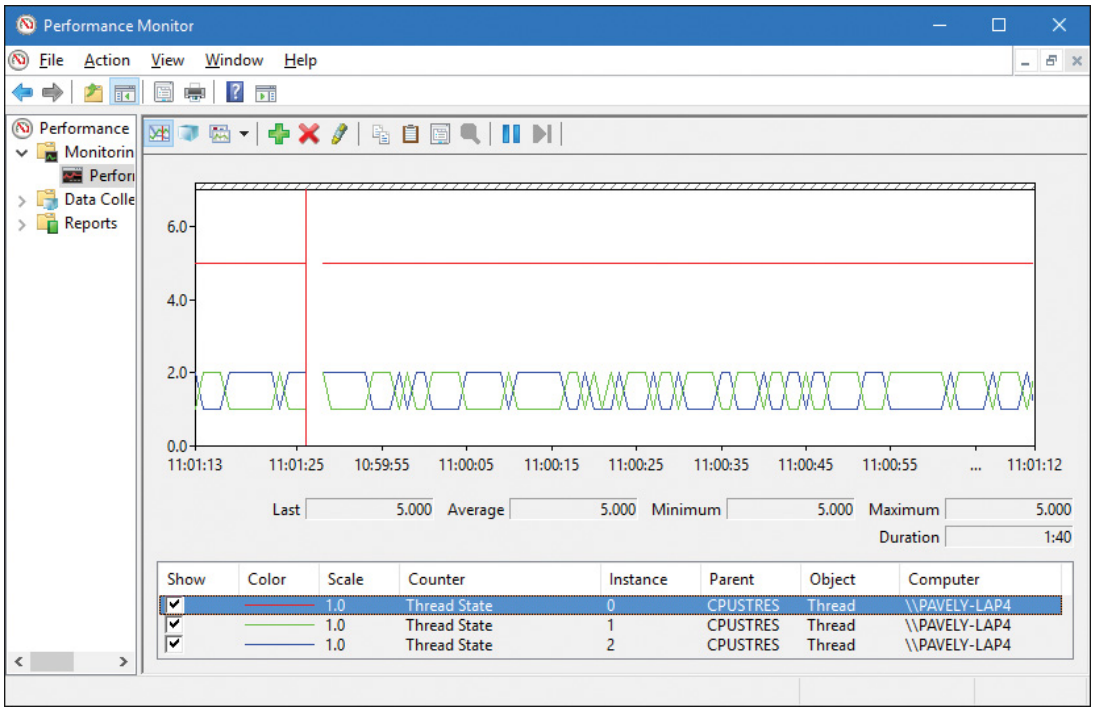
more often than thread 1:



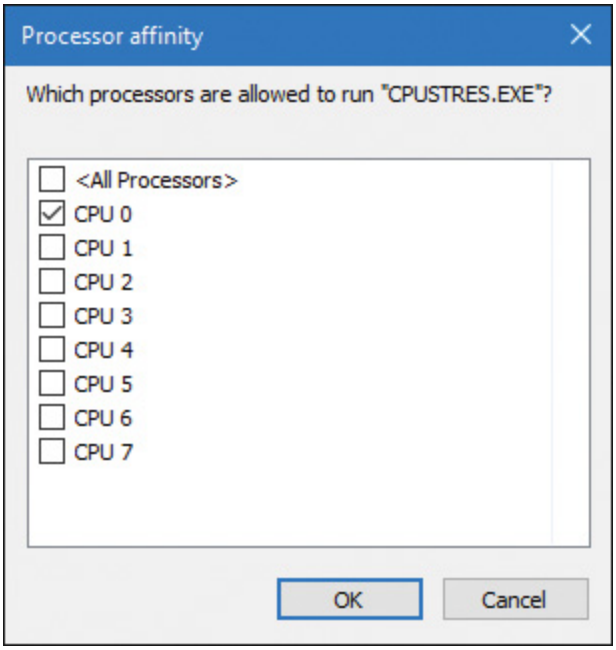
14. Right-click thread 1 and choose an activity level of **Maximum**. Then repeat this step for thread 2. Both threads now should be constantly in state 2 because they're running essentially an infinite loop:



If you're trying this on a single processor system, you'll see something different. Because there is only one processor, only one thread can execute at a time, so you'll see the two threads alternating between states 1 (ready) and 2 (running):

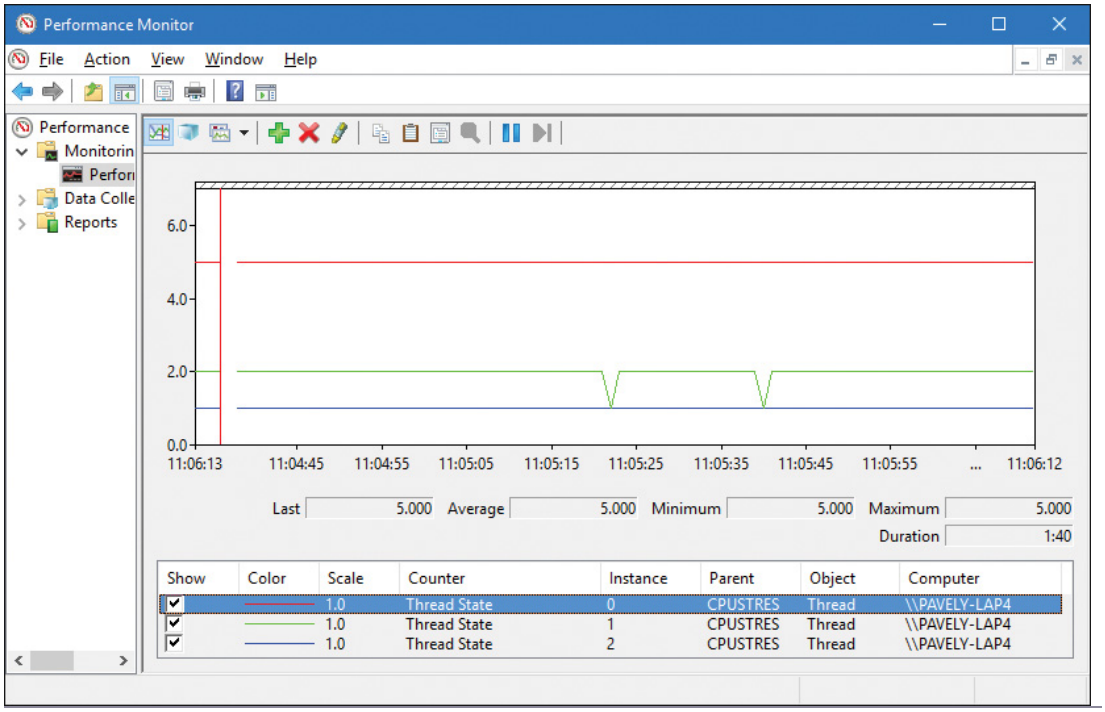


15. If you're on a multiprocessor system (very likely), you can get the same effect by going to Task Manager, right-clicking the CPUSTRES process, selecting **Set Affinity**, and then select just one processor—it doesn't matter which one—as shown here. (You can also do it from CPU Stress by opening the Process menu and selecting **Affinity**.)



16. There's one more thing you can try. With this setting in place, go back to CPU Stress, right-click thread 1, and choose a priority of **Above Normal**. You'll see that thread 1 is running continuously (state 2) and thread 2 is always in the ready state (state 1). This is because there's only one processor, so in general, the higher priority thread wins out. From time to time, however, you'll see a change in thread 1's state to

ready. This is because every 4 seconds or so, the starved thread gets a boost that enables it to run for a little while. (Often, this state change is not reflected by the graph because the granularity of Performance Monitor is limited to 1 second, which is too coarse.) This is described in more detail later in this chapter in the section "**Priority boosts**."



Dispatcher database

To make thread-scheduling decisions, the kernel maintains a set of data structures known collectively as the *dispatcher database*. The dispatcher database keeps track of which threads are waiting to execute and which processors are executing which threads.

To improve scalability, including thread-dispatching concurrency, Windows multiprocessor systems have per-processor dispatcher ready queues and shared processor group queues, as illustrated in **Figure 4-9**. In this way, each CPU can check its own shared ready queue for the next thread to run without having to lock the system-wide ready queues.

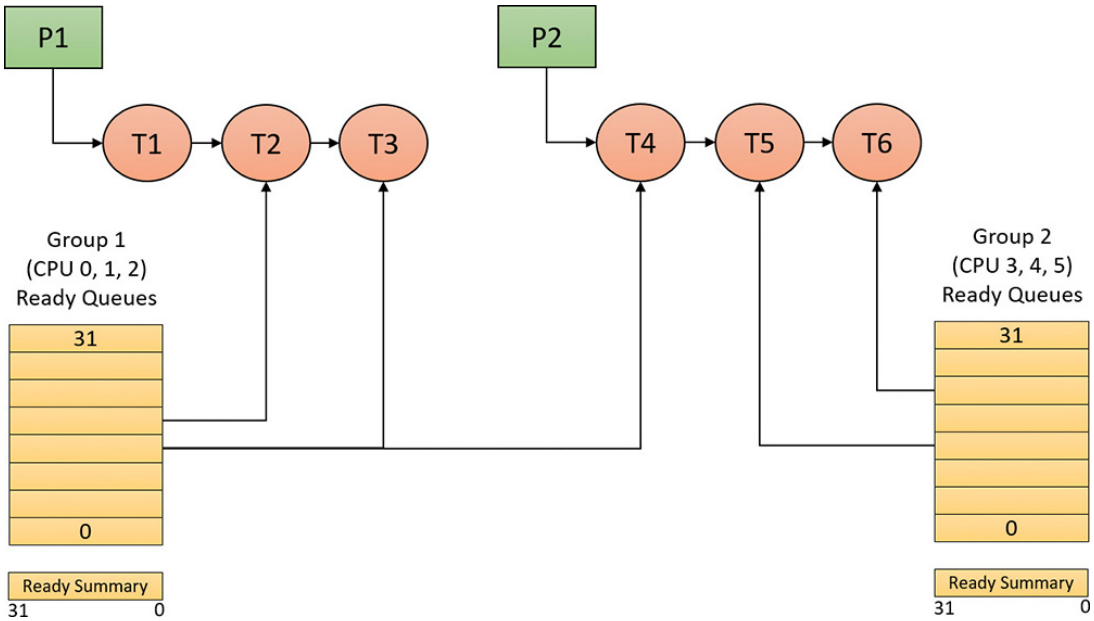


FIGURE 4-9 Windows multiprocessor dispatcher database. (This example shows six processors. *P* represents processes; *T* represents threads.)

Windows versions prior to Windows 8 and Windows Server 2012 used per-processor ready queues and a per-processor ready summary, which were stored as part of processor control block (PRCB) structure. (To see the fields in the PRCB, type `dt nt!_kprcb` in the kernel debugger.) Starting with Windows 8 and Windows Server 2012, a shared ready queue and ready summary are used for a group of processors. This enables the system to make better decisions about which processor to use next for that group of processors. (The per-CPU ready queues are still there and used for threads with affinity constraints.)

NOTE

Because the shared data structure must be protected (by a spinlock), the group should not be too large. That way, contention on the queues is insignificant. In the current implementation, the maximum group size is four logical processors. If the number of logical processors is greater than four, then more than one group would be created, and the available processors spread evenly. For example, on a six-processor system, two groups of three processors each would be created.

The ready queues, ready summary (described next), and some other information is stored in a kernel structure named `KSHARED_READY_QUEUE` that is stored in the PRCB. Although it exists for every processor, it's used only on the first processor of each processor group, sharing it with the rest of the processors in that group.

The dispatcher ready queues (`ReadListHead` in `KSHARED_READY_QUEUE`) contain the threads that are in the ready state, waiting to be scheduled for execution. There is one queue for each of the 32 priority levels. To speed up the selection of which thread to run or preempt, Windows maintains a 32-bit bitmask called the *ready summary* (`ReadySummary`). Each bit set indicates one or more threads in the ready queue for that priority level (bit 0 represents priority 0, bit 1 priority 1, and so on).

Instead of scanning each ready list to see whether it is empty or not (which would make scheduling decisions dependent on the number of different priority threads), a single bit scan is performed as a native processor command to find the highest bit set. Regardless of the number of threads in the ready queue, this operation takes a constant amount of time.

The dispatcher database is synchronized by raising IRQL to `DISPATCH_LEVEL (2)`. (For an explanation of interrupt priority levels, or IRQLs, see **Chapter 6**.) Raising IRQL in this way prevents other

threads from interrupting thread dispatching on the processor because threads normally run at IRQL 0 or 1. However, more is required than just raising IRQL, because other processors can simultaneously raise to the same IRQL and attempt to operate on their dispatcher database. How Windows synchronizes access to the dispatcher database is explained later in this chapter in the section "**Multiprocessor systems**."

EXPERIMENT: VIEWING READY THREADS

You can view the list of ready threads with the kernel-debugger `!ready` command. This command displays the thread or list of threads that are ready to run at each priority level. Here is an example generated on a 32-bit machine with four logical processors:

Click here to view code image

```
0: kd> !ready
KSHARED_READY_QUEUE 8147e800: (00) ****----------------------------
SharedReadyQueue 8147e800: Ready Threads at priority 8
   THREAD 80af8bc0  Cid 1300.15c4  Teb: 7ffdb000 Win32Thread:
00000000 READY on
processor 80000002
   THREAD 80b58bc0  Cid 0454.0fc0  Teb: 7f82e000 Win32Thread:
00000000 READY on
processor 80000003
SharedReadyQueue 8147e800: Ready Threads at priority 7
   THREAD a24b4700  Cid 0004.11dc  Teb: 00000000 Win32Thread:
00000000 READY on
processor 80000001
   THREAD a1bad040  Cid 0004.096c  Teb: 00000000 Win32Thread:
00000000 READY on
processor 80000001
SharedReadyQueue 8147e800: Ready Threads at priority 6
   THREAD a1bad4c0  Cid 0004.0950  Teb: 00000000 Win32Thread:
00000000 READY on
processor 80000002
   THREAD 80b5e040  Cid 0574.12a4  Teb: 7fc33000 Win32Thread:
00000000 READY on
processor 80000000
SharedReadyQueue 8147e800: Ready Threads at priority 4
   THREAD 80b09bc0  Cid 0004.12dc  Teb: 00000000 Win32Thread:
00000000 READY on
processor 80000003
SharedReadyQueue 8147e800: Ready Threads at priority 0
```

THREAD 82889bc0 Cid 0004.0008 Teb: 00000000 Win32Thread: 00000000 READY on
processor 80000000
Processor 0: No threads in READY state
Processor 1: No threads in READY state
Processor 2: No threads in READY state
Processor 3: No threads in READY state

The processor numbers have a `0x8000000` added to them, but the actual processor numbers are easy to see. The first line shows the address of the `KSHARED_READY_QUEUE` with the group number in parentheses (`00` in the output) and then a graphic representation of the processors in this particular group (the four asterisks).

The last four lines seem odd, as they appear to indicate no ready threads, contradicting the preceding output. These lines indicate ready threads from the older `DispatcherReadyListHead` member of the PRCB because the per-processor ready queues are used for threads that have restrictive affinity (set to run on a subset of processors inside that processor group).

You can also dump the `KSHARED_READY_QUEUE` with the address given by the `!ready` command:

Click here to view code image

```
0: kd> dt nt!_KSHARED_READY_QUEUE 8147e800
   +0x000 Lock           : 0
   +0x004 ReadySummary   : 0x1d1
   +0x008 ReadyListHead  : [32] _LIST_ENTRY [ 0x82889c5c - 0x82889c5c ]
   +0x108 RunningSummary : [32]  "???"
   +0x128 Span           : 4
   +0x12c LowProcIndex   : 0
   +0x130 QueueIndex     : 1
   +0x134 ProcCount      : 4
   +0x138 Affinity       : 0xf
```

The `ProcCount` member shows the processor count in the shared group (`4` in this example). Also note the `ReadySummary` value, `0x1d1`. This translates to 111010001 in binary. Reading the binary one bits from right to left, this indicates that threads exist in priorities 0, 4, 6, 7, 8, which match the preceding output.

Quantum

As mentioned earlier in the chapter, a *quantum* is the amount of time a thread is permitted to run before Windows checks to see whether another thread at the same priority is waiting to run. If a thread completes its quantum and there are no other threads at its priority, Windows permits the thread to run for another quantum.

On client versions of Windows, threads run for two clock intervals by default. On server systems, threads runs for 12 clock intervals by default. (We'll explain how to change these values in the "**Controlling the quantum**" section.) The rationale for the longer default value on server systems is to minimize context switching. By having a longer quantum, server applications that wake up because of a client request have a better chance of completing the request and going back into a wait state before their quantum ends.

The length of the clock interval varies according to the hardware platform. The frequency of the clock interrupts is up to the HAL, not the kernel. For example, the clock interval for most x86 uniprocessors is about 10 milliseconds (note that these machines are no longer supported by Windows and are used here only for example purposes), and for most x86 and x64 multiprocessors it is about 15 milliseconds. This clock interval is stored in the kernel variable `KeMaximumIncrement` as hundreds of nanoseconds.

Although threads run in units of clock intervals, the system does not use the count of clock ticks as the gauge for how long a thread has run and whether its quantum has expired. This is because thread run-time

accounting is based on processor cycles. When the system starts up, it multiplies the processor speed (CPU clock cycles per second) in hertz (Hz) by the number of seconds it takes for one clock tick to fire (based on the `KeMaximumIncrement` value described earlier) to calculate the number of clock cycles to which each quantum is equivalent. This value is stored in the kernel variable `KiCyclesPerClockQuantum`.

The result of this accounting method is that threads do not actually run for a quantum number based on clock ticks. Instead, they run for a quantum target, which represents an estimate of what the number of CPU clock cycles the thread has consumed should be when its turn would be given up. This target should be equal to an equivalent number of clock interval timer ticks. This is because, as you just saw, the calculation of clock cycles per quantum is based on the clock interval timer frequency, which you can check using the following experiment. Note, however, that because interrupt cycles are not charged to the thread, the actual clock time might be longer.

EXPERIMENT: DETERMINING THE CLOCK INTERVAL FREQUENCY

The Windows `GetSystemTimeAdjustment` function returns the clock interval. To determine the clock interval, run the `clockres` tool from Sysinternals. Here's the output from a quad-core 64-bit Windows 10 system:

Click here to view code image

```
C:\>clockres

ClockRes v2.0 - View the system clock resolution
Copyright (C) 2009 Mark Russinovich
SysInternals - www.sysinternals.com

Maximum timer interval: 15.600 ms
Minimum timer interval: 0.500 ms
Current timer interval: 1.000 ms
```

The current interval may be lower than the maximum (default) clock interval because of multimedia timers. Multimedia timers are used with functions such as `timeBeginPeriod` and `timeSetEvent` that are used to receive callbacks with intervals of 1 millisecond (ms) at best. This causes a global reprogramming of the kernel interval timer, meaning the scheduler wakes up in more frequent intervals, which can degrade system performance. In any case, this does not affect quantum lengths, as described in the next section.

It's also possible to read the value using the kernel global variable `KeMaximumIncrement` as shown here (not the same system as the previous example):

Click here to view code image

```
0: kd> dd nt!KeMaximumIncrement L1
814973b4  0002625a
0: kd> ? 0002625a
Evaluate expression: 156250 = 0002625a
```

This corresponds to the default of 15.6 ms.

Quantum accounting

Each process has a quantum reset value in the process control block (`KPROCESS`). This value is used when creating new threads inside the process and is duplicated in the thread control block (`KTHREAD`), which is then used when giving a thread a new quantum target. The quantum reset value is stored in terms of actual quantum units (we'll discuss what these mean soon), which are then multiplied by the number of clock cycles per quantum, resulting in the quantum target.

As a thread runs, CPU clock cycles are charged at different events, such as context switches, interrupts, and certain scheduling decisions. If, at a clock interval timer interrupt, the number of CPU clock cycles charged has reached (or passed) the quantum target, quantum end processing is triggered. If there is another thread at the same priority waiting to run, a context switch occurs to the next thread in the ready queue.

Internally, a quantum unit is represented as one-third of a clock tick. That is, one clock tick equals three quantums. This means that on client Windows systems, threads have a quantum reset value of 6 (2 * 3) and that server systems have a quantum reset value of 36 (12 * 3) by default. For this reason, the `KiCyclesPerClockQuantum` value is divided by 3 at the end of the calculation previously described, because the original value describes only CPU clock cycles per clock interval timer tick.

The reason a quantum was stored internally as a fraction of a clock tick rather than as an entire tick was to allow for partial quantum decay-on-wait completion on versions of Windows prior to Windows Vista. Prior versions used the clock interval timer for quantum expiration. If this adjustment had not been made, it would have been possible for threads to never have their quantums reduced. For example, if a thread ran, entered a wait state, ran again, and entered another wait state but was

never the currently running thread when the clock interval timer fired, it would never have its quantum charged for the time it was running. Because threads now have CPU clock cycles charged instead of quantums, and because this no longer depends on the clock interval timer, these adjustments are not required.

EXPERIMENT: DETERMINING THE CLOCK CYCLES PER QUANTUM

Windows doesn't expose the number of clock cycles per quantum through any function. However, with the calculation and description we've given, you should be able to determine this on your own using the following steps and a kernel debugger such as WinDbg in local debugging mode:

1. Obtain your processor frequency as Windows has detected it. You can use the value stored in the PRCB's MHz field, which you can display with the `!cpuinfo` command. Here is a sample output of a four-processor system running at 2794 megahertz (MHz):

Click here to view code image

```
lkd> !cpuinfo

CP  F/M/S Manufacturer  MHz PRCB Signature    MSR 8B Signature Features
 0  6,60,3 GenuineIntel 2794 ffffffff00000000 >ffffffff00000000<a3cd3fff
 1  6,60,3 GenuineIntel 2794 ffffffff00000000               a3cd3fff
 2  6,60,3 GenuineIntel 2794 ffffffff00000000               a3cd3fff
 3  6,60,3 GenuineIntel 2794 ffffffff00000000               a3cd3fff
```

2. Convert the number to hertz (Hz). This is the number of CPU clock cycles that occur each second on your system—in this case, 2,794,000,000 cycles per second.

3. Obtain the clock interval on your system by using `clockres`. This measures how long it takes before the clock fires. On the sample system used here, this interval was 15.625 msec.

4. Convert this number to the number of times the clock interval timer fires each second. One second equals 1,000 ms, so divide the number derived in step 3 by 1,000. In this case, the timer fires every 0.015625 seconds.

5. Multiply this count by the number of cycles each second that you obtained in step 2. In this case, 43,656,250 cycles have elapsed after each clock interval.

6. Remember that each quantum unit is one-third of a clock interval, so divide the number of cycles by 3. This gives you 14,528,083, or 0xDE0C13 in hexadecimal. This is the number of clock cycles each quantum unit should take on a system running at 2,794 MHz with a clock interval of around 15.6 ms.

7. To verify your calculation, dump the value of `KiCyclesPerClockQuantum` on your system. It should match (or be close enough because of rounding errors).

Click here to view code image

```
lkd> dd nt!KiCyclesPerClockQuantum L1
8149755c  00de0c10
```

Controlling the quantum

You can change the thread quantum for all processes, but you can choose only one of two settings: short (two clock ticks, which is the default for client machines) or long (12 clock ticks, which is the default for server systems).

NOTE

By using the job object on a system running with long quantums, you can select other quantum values for the processes in the job.

To change this setting, right-click the **This PC** icon on the desktop. Alternatively, in Windows Explorer, choose **Properties**, click the **Advanced System Settings** label, click the **Advanced** tab, click the

Settings button in the **Performance** section, and click yet another **Advanced** tab. **Figure 4-10** shows the resulting dialog box.

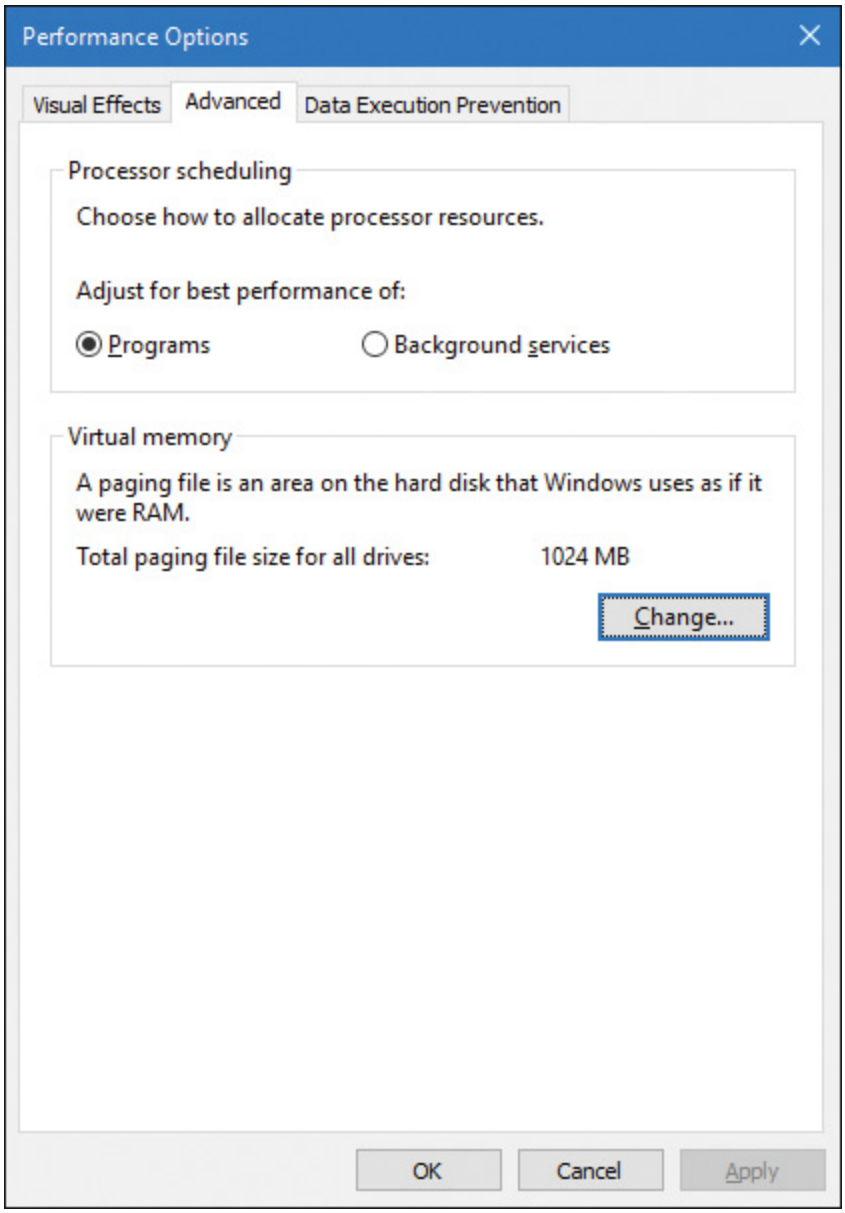


FIGURE 4-10 Quantum configuration in the Performance Options dialog box.

This dialog box contains two key options:

■ **Programs** This setting designates the use of short, variable quantums, which is the default for client versions of Windows (and other client-like versions, such as mobile, XBOX, HoloLens, and so on). If you install Terminal Services on a server system and configure the server as an application server, this setting is selected so that the users on the terminal server have the same quantum settings that would normally be set on a

desktop or client system. You might also select this manually if you were running Windows Server as your desktop OS.

■ **Background Services** This setting designates the use of long, fixed quantums—the default for server systems. The only reason you might select this option on a workstation system is if you were using the workstation as a server system. However, because changes in this option take effect immediately, it might make sense to use it if the machine is about to run a background or server-style workload. For example, if a long-running computation, encoding, or modeling simulation needs to run overnight, you could select the Background Services option at night and return the system to Programs mode in the morning.

Variable quantums

When variable quantums are enabled, the variable quantum table (`PspVariableQuantums`), which holds an array of six quantum numbers, is loaded into the `PspForegroundQuantum` table (a three-element array) that is used by the `PspComputeQuantum` function. Its algorithm will pick the appropriate quantum index based on whether the process is a foreground process—that is, whether it contains the thread that owns the foreground window on the desktop. If this is not the case, an index of 0 is chosen, which corresponds to the default thread quantum described earlier. If it is a foreground process, the quantum index corresponds to the priority separation.

This priority separation value determines the priority boost (described in the upcoming section "**Priority boosts**") that the scheduler will apply to foreground threads, and it is thus paired with an appropriate extension of the quantum. For each extra priority level (up to 2), another quantum is given to the thread. For example, if the thread receives a boost of one priority level, it receives an extra quantum as well. By default, Windows sets the maximum possible priority boost to foreground threads, meaning that the priority separation will be 2, which means quantum index 2 is selected in the variable quantum table. This leads to the thread receiving two extra quantums, for a total of three quantums.

Table 4-2 describes the exact quantum value (recall that this is stored in a unit representing one-third of a clock tick) that will be selected based on the quantum index and which quantum configuration is in use.

	Short Quantum Index			Long Quantum Index		
Variable	6	12	18	12	24	36
Fixed	18	18	18	36	36	36

TABLE 4-2 Quantum values

Thus, when a window is brought into the foreground on a client system, all the threads in the process containing the thread that owns the foreground window have their quantums tripled. Threads in the foreground process run with a quantum of six clock ticks, whereas threads in other processes have the default client quantum of two clock ticks. In this way, when you switch away from a CPU-intensive process, the new foreground process will get proportionally more of the CPU. This is because when its threads run, they will have a longer turn than background threads (again, assuming the thread priorities are the same in both the foreground and background processes).

Quantum settings registry value

The user interface that controls quantum settings described earlier modifies the registry value `Win32-PrioritySeparation` in the key `HKLM\SYSTEM\CurrentControlSet\Control\PriorityControl`. In addition to specifying the relative length of thread quantums (short or long), this registry value also defines whether variable quantums should be used, as well as the priority separation (which, as you've seen, will determine the quantum index used when variable quantums are enabled). This value consists of 6 bits divided into the three 2-bit fields shown in Figure 4-11.

FIGURE 4-11 Fields of the Win32PrioritySeparation registry value.

The fields shown in **Figure 4-11** can be defined as follows:

■ **Short vs. Long** A value of 1 specifies long quantums, and a value of 2 specifies short ones. A setting of 0 or 3 indicates that the default appropriate for the system will be used (short for client systems, long for server systems).

■ **Variable vs. Fixed** A value of 1 means to enable the variable quantum table based on the algorithm shown in the "**Variable quantums**" section. A setting of 0 or 3 means that the default appropriate for the system will be used (variable for client systems, fixed for server systems).

■ **Priority Separation** This field (stored in the kernel variable `PsPrioritySeparation`) defines the priority separation (up to 2), as explained in the "**Variable quantums**" section.

When you use the Performance Options dialog box (refer to **Figure 4-10**), you can choose from only two combinations: short quantums with foreground quantums tripled, or long quantums with no quantum changes for foreground threads. However, you can select other combinations by modifying the `Win32PrioritySeparation` registry value directly.

Threads that are part of a process running in the idle process priority class always receive a single thread quantum, ignoring any sort of quantum configuration settings, whether set by default or set through the registry.

On Windows Server systems configured as application servers, the initial value of the `Win32PrioritySeparation` registry value will be hex 26, which is identical to the value set by the Optimize Performance for Programs option in the Performance Options dialog box. This selects quantum and priority-boost behavior like that on Windows client systems, which is appropriate for a server used primarily to host users' applications.

On Windows client systems and on servers not configured as application servers, the initial value of the `Win32PrioritySeparation` registry setting will be 2. This provides values of 0 for the Short vs. Long and Variable vs. Fixed Bit fields, relying on the default behavior of the system (depending on whether it is a client system or a server system) for these options. However, it provides a value of 2 for the Priority Separation field. After the registry value has been changed via the Performance Options dialog box, it cannot be restored to this original value other than by modifying the registry directly.

EXPERIMENT: EFFECTS OF CHANGING THE QUANTUM CONFIGURATION

Using a local kernel debugger, you can see how the two quantum configuration settings, Programs and Background Services, affect the `PsPrioritySeparation` and `PspForegroundQuantum` tables, as well as modify the `QuantumReset` value of threads on the system. Take the following steps:

1. Open the **System** utility in Control Panel or right-click the **This PC** icon on the desktop and choose **Properties**.

2. Click the **Advanced System Settings** label, click the **Advanced** tab, click the **Settings** button in the Performance section, and click the second **Advanced** tab.

3. Select the **Programs** option and click **Apply**. Keep this dialog box open for the duration of the experiment.

4. Dump the values of `PsPrioritySeparation` and `PspForegroundQuantum`, as shown here. The values shown are what you should see on a Windows system after making the change in steps 1–3. Notice how the variable short quantum table is being used and that a priority boost of 2 will apply to foreground applications:

Click here to view code image

```
lkd> dd nt!PsPrioritySeparation L1
fffff803'75e0e388  00000002
lkd> db nt!PspForegroundQuantum L3

fffff803'76189d28  06 0c 12
```

5. Look at the `QuantumReset` value of any process on the system. As noted, this is the default full quantum of each thread on the system when it is replenished. This value is cached into each thread of the process, but the `KPROCESS` structure is easier to look at. Notice in this case it is 6, because WinDbg, like most other applications, gets the quantum set in the first entry of the `PspForegroundQuantum` table:

Click here to view code image

```
lkd> .process
Implicit process is now ffffe001'4f51f080
lkd> dt nt!_KPROCESS ffffe001'4f51f080 QuantumReset
   +0x1bd QuantumReset : 6 ''
```

6. Change the Performance option to **Background Services** in the dialog box you opened in steps 1 and 2.

7. Repeat the commands shown in steps 4 and 5. You should see the values change in a manner consistent with our discussion in this section:

Click here to view code image

```
lkd> dd nt!PsPrioritySeparation L1
fffff803'75e0e388  00000000
lkd> db nt!PspForegroundQuantum L3
fffff803'76189d28  24 24 24
lkd> dt nt!_KPROCESS ffffe001'4f51f080 QuantumReset
   +0x1bd QuantumReset : 36 '$'
```

Priority boosts

The Windows scheduler periodically adjusts the current (dynamic) priority of threads through an internal priority-boosting mechanism. In many cases, it does so to decrease various latencies (that is, to make threads respond faster to the events they are waiting on) and increase responsiveness. In others, it applies these boosts to prevent inversion and starvation scenarios. Here are some of the boost scenarios that will be described in this section (and their purpose):

- Boosts due to scheduler/dispatcher events (latency reduction)
- Boosts due to I/O completion (latency reduction)

- Boosts due to user interface (UI) input (latency reduction/responsiveness)

- Boosts due to a thread waiting on an executive resource (`ERESOURCE`) for too long (starvation avoidance)

- Boosts when a thread that's ready to run hasn't been running for some time (starvation and priority-inversion avoidance)

Like any scheduling algorithms, however, these adjustments aren't perfect, and they might not benefit all applications.

NOTE

Windows never boosts the priority of threads in the real-time range (16 through 31). Therefore, scheduling is always predictable with respect to other threads in this range. Windows assumes that if you're using the real-time thread priorities, you know what you're doing.

Client versions of Windows also include a pseudo-boosting mechanism that occurs during multimedia playback. Unlike the other priority boosts, multimedia-playback boosts are managed by a kernel-mode driver called the Multimedia Class Scheduler Service (mmcss.sys). They are not really boosts, however. The driver merely sets new priorities for the threads as needed. Therefore, none of the rules regarding boosts apply. We'll first cover the typical kernel-managed priority boosts and then talk about MMCSS and the kind of "boosting" it performs.

Boosts due to scheduler/dispatcher events

Whenever a dispatch event occurs, the `KiExitDispatcher` routine is called. Its job is to process the deferred ready list by calling `KiProcessThreadWaitList` and then call `KzCheckForThreadDispatch` to check whether any threads on the current processor should not be scheduled. Whenever such an event occurs, the caller can also specify which type of boost should be applied to the thread, as well as what priority increment the boost should be associated with. The following scenarios are considered as `AdjustUnwait` dispatch events because they deal with a dispatcher (synchronization) object entering a signaled state, which might cause one or more threads to wake up:

■ An asynchronous procedure call (APC; described in **Chapter 6** and in more detail in Chapter 8 in Part 2) is queued to a thread.

■ An event is set or pulsed.

■ A timer was set, or the system time was changed, and timers had to be reset.

■ A mutex was released or abandoned.

■ A process exited.

■ An entry was inserted in a queue (`KQUEUE`), or the queue was flushed.

■ A semaphore was released.

■ A thread was alerted, suspended, resumed, frozen, or thawed.

■ A primary UMS thread is waiting to switch to a scheduled UMS thread.

For scheduling events associated with a public API (such as `SetEvent`), the boost increment applied is specified by the caller. Windows recommends certain values to be used by developers, which will be described

later. For alerts, a boost of 2 is applied (unless the thread is put in an alert wait by calling `KeAlertThreadByThreadId`, in which case the applied boost is 1), because the alert API does not have a parameter allowing a caller to set a custom increment.

The scheduler also has two special `AdjustBoost` dispatch events, which are part of the lock-ownership priority mechanism. These boosts attempt to fix situations in which a caller that owns the lock at priority *x* ends up releasing the lock to a waiting thread at priority < = *x*. In this situation, the new owner thread must wait for its turn (if running at priority *x*), or worse, it might not even get to run at all if its priority is lower than *x*. This means the releasing thread continues its execution, even though it should have caused the new owner thread to wake up and take control of the processor. The following two dispatcher events cause an `AdjustBoost` dispatcher exit:

- An event is set through the `KeSetEventBoostPriority` interface, which is used by the `ERESOURCE` reader-writer kernel lock.

- A gate is set through the `KeSignalGate` interface, which is used by various internal mechanisms when releasing a gate lock.

Unwait boosts

Unwait boosts attempt to decrease the latency between a thread waking up due to an object being signaled (thus entering the ready state) and the thread actually beginning its execution to process the unwait (thus entering the running state). Generally speaking, it is desirable that a thread that wakes up from a waiting state would be able to run as soon as possible.

The various Windows header files specify recommended values that kernel-mode callers of APIs such as `KeReleaseMutex`, `KeSetEvent` and `KeReleaseSemaphore` should use, which correspond to definitions such as `MUTANT_INCREMENT`, `SEMAPHORE_INCREMENT`, and `EVENT_INCREMENT`. These three definitions have always been set to 1 in the headers, so it is safe to assume that most unwaits on these objects result in a boost of 1.

In the user-mode API, an increment cannot be specified, nor do the native system calls such as `NtSetEvent` have parameters to specify such a boost. Instead, when these APIs call the underlying `Ke` interface, they automatically use the default `_INCREMENT` definition. This is also the case when mutexes are abandoned or timers are reset due to a system time change: The system uses the default boost that normally would have been applied when the mutex would have been released. Finally, the APC boost is completely up to the caller. Soon, you'll see a specific usage of the APC boost related to I/O completion.

NOTE

Some dispatcher objects don't have boosts associated with them. For example, when a timer is set or expires, or when a process is signaled, no boost is applied.

All these boosts of 1 attempt to solve the initial problem by assuming that both the releasing and waiting threads are running at the same priority. By boosting the waiting thread by one priority level, the waiting thread should preempt the releasing thread as soon as the operation completes. Unfortunately, on uniprocessor systems, if this assumption does not hold, the boost might not do much. For example, if the waiting thread is at priority 4 and the releasing thread is at priority 8, waiting at priority 5 won't do much to reduce latency and force preemption. On multiprocessor systems, however, due to the stealing and balancing algorithms, this higher-priority thread may have a better chance of getting picked up by another logical processor. This is due to a design choice made in the initial NT architecture, which is to not track lock ownership (except a few locks). This means the scheduler can't be sure who really owns an event and if it's really being used as a lock. Even with lock-ownership tracking, ownership is not usually passed (to avoid convoy issues) other than in the executive resource case, explained in an upcoming section.

For certain kinds of lock objects that use events or gates as their underlying synchronization object, the lock-ownership boost resolves the

dilemma. Also, on a multiprocessor machine, the ready thread might get picked up on another processor (due to the processor-distribution and load-balancing schemes you'll see later), and its high priority might increase the chances of it running on that secondary processor instead.

Lock-ownership boosts

Because the executive-resource (`ERESOURCE`) and critical-section locks use underlying dispatcher objects, releasing these locks results in an unwait boost as described earlier. On the other hand, because the high-level implementation of these objects tracks the owner of the lock, the kernel can make a more informed decision as to what kind of boost should be applied by using the `AdjustBoost` reason. In these kinds of boosts, `AdjustIncrement` is set to the current priority of the releasing (or setting) thread, minus any graphical user interface (GUI) foreground separation boost. In addition, before the `KiExitDispatcher` function is called, `KiRemoveBoostThread` is called by the event and gate code to return the releasing thread back to its regular priority. This step is needed to avoid a lock-convoy situation, in which two threads repeatedly passing the lock between one another get ever-increasing boosts.

NOTE

Pushlocks, which are unfair locks because ownership of the lock in a contended acquisition path is not predictable (rather, it's random, like a spinlock), do not apply priority boosts due to lock ownership. This is because doing so only contributes to preemption and priority proliferation, which isn't required because the lock becomes immediately free as soon as it is released (bypassing the normal wait/unwait path).

Other differences between the lock-ownership boost and unwait boost will be exposed in the way the scheduler actually applies boosting, which is the subject of the next section.

Priority boosting after I/O completion

Windows gives temporary priority boosts upon completion of certain I/O operations so that threads that were waiting for an I/O have more of a chance to run right away and process whatever was being waited for. Although you'll find recommended boost values in the Windows Driver Kit (WDK) header files (by searching for *#define IO_* in Wdm.h or Ntddk.h), the actual value for the boost is up to the device driver. (These values are listed in **Table 4-3**.) It is the device driver that specifies the boost when it completes an I/O request on its call to the kernel function, `IoCompleteRequest`. In **Table 4-3**, notice that I/O requests to devices that warrant better responsiveness have higher boost values.

Device	Boost
Disk, CD-ROM, parallel, video	1
Network, mailslot, named pipe, serial	2
Keyboard, mouse	6
Sound	8

TABLE 4-3 Recommended boost values

NOTE

You might intuitively expect better responsiveness from your video card or disk than a boost of 1. However, the kernel is in fact trying to optimize for *latency*, to which some devices (as well as human sensory inputs) are more sensitive than others. To give you an idea, a sound card expects data around every 1 ms to play back music without perceptible glitches, while a video card needs to output at only 24 frames per second, or about once every 40 ms, before the human eye can notice glitches.

As hinted earlier, these I/O completion boosts rely on the unwait boosts seen in the previous section. **Chapter 6** shows the mechanism of I/O completion in depth. For now, the important detail is that the kernel im-

plements the signaling code in the `IoCompleteRequest` API through the use of either an APC (for asynchronous I/O) or through an event (for synchronous I/O). When a driver passes in—for example, `IO_DISK_INCREMENT` to `IoCompleteRequest` for an asynchronous disk read—the kernel calls `KeInsertQueueApc` with the boost parameter set to `IO_DISK_INCREMENT`. In turn, when the thread's wait is broken due to the APC, it receives a boost of 1.

Be aware that the boost values given in **Table 4-3** are merely recommendations by Microsoft. Driver developers are free to ignore them, and certain specialized drivers can use their own values. For example, a driver handling ultrasound data from a medical device, which must notify a user-mode visualization application of new data, would probably use a boost value of 8 as well, to satisfy the same latency as a sound card. In most cases, however, due to the way Windows driver stacks are built (again, see **Chapter 6** for more information), driver developers often write *minidrivers*, which call into a Microsoft-owned driver that supplies its own boost to `IoCompleteRequest`. For example, RAID or SATA controller card developers typically call `StorPortCompleteRequest` to complete processing their requests. This call does not have any parameter for a boost value, because the Storport.sys driver fills in the right value when calling the kernel. Additionally, whenever any file system driver (identified by setting its device type to `FILE_DEVICE_DISK_FILE_SYSTEM` or `FILE_DEVICE_NETWORK_FILE_SYSTEM`) completes its request, a boost of `IO_DISK_INCREMENT` is always applied if the driver passed in `IO_NO_INCREMENT (0)` instead. So this boost value has become less of a recommendation and more of a requirement enforced by the kernel.

Boosts during waiting on executive resources

When a thread attempts to acquire an executive resource (`ERESOURCE`; see Chapter 8 in Part 2 for more information on kernel-synchronization objects) that is already owned exclusively by another thread, it must enter a wait state until the other thread has released the resource. To limit the risk of deadlocks, the executive performs this wait in intervals of 500 ms instead of doing an infinite wait on the resource. At the end of these 500 ms, if the resource is still owned, the executive attempts to prevent CPU starvation by acquiring the dispatcher lock, boosting the owning thread or threads to 15 (if the original owner priority is less than the waiter's and not already 15), resetting their quantums, and performing another wait.

Because executive resources can be either shared or exclusive, the kernel first boosts the exclusive owner and then checks for shared owners and boosts all of them. When the waiting thread enters the wait state again, the hope is that the scheduler will schedule one of the owner threads, which will have enough time to complete its work and release the resource. Note that this boosting mechanism is used only if the resource doesn't have the Disable Boost flag set, which developers can choose to set if the priority-inversion mechanism described here works well with their usage of the resource.

Additionally, this mechanism isn't perfect. For example, if the resource has multiple shared owners, the executive boosts all those threads to priority 15. This results in a sudden surge of high-priority threads on the system, all with full quantums. Although the initial owner thread will run first (because it was the first to be boosted and therefore is first on the ready list), the other shared owners will run next because the waiting thread's priority was not boosted. Only after all the shared owners have had a chance to run and their priority has been decreased below the waiting thread will the waiting thread finally get its chance to acquire the resource. Because shared owners can promote or convert their ownership from shared to exclusive as soon as the exclusive owner releases the resource, it's possible for this mechanism not to work as intended.

Priority boosts for foreground threads after waits

As will be described shortly, whenever a thread in the foreground process completes a wait operation on a kernel object, the kernel boosts its current (not base) priority by the current value of `PsPrioritySeparation`. (The windowing system is responsible for determining which process is considered to be in the foreground.) As described earlier in this chapter in the section "**Controlling the quantum**," `PsPrioritySeparation` reflects the quantum-table index used to select quantums for the threads of foreground applications. However, in this case, it is being used as a priority boost value.

The reason for this boost is to improve the responsiveness of interactive applications. By giving the foreground application a small boost when it completes a wait, it has a better chance of running right away, especially when other processes at the same base priority might be running in the background.

EXPERIMENT: WATCHING FOREGROUND PRIORITY BOOSTS AND DECAYS

Using the CPU Stress tool, you can watch priority boosts in action. Take the following steps:

1. Open the System utility in Control Panel or right-click the **This Computer** icon on the desktop and choose **Properties**.

2. Click the **Advanced System Settings** label, click the **Advanced** tab, click the **Settings** button in the Performance section, and click the **Advanced** tab.

3. Select the **Programs** option. This gives `PsPrioritySeparation` a value of *2*.

4. Run CPU Stress, right-click thread 1, and choose **Busy** from the context menu.

5. Start the Performance Monitor tool.

6. Click the **Add Counter** toolbar button or press **Ctrl+I** to open the Add Counters dialog box.

7. Select the **Thread** object and then select the **Priority Current** counter.

8. In the Instances box, select <**All Instances**> and click **Search**.

9. Scroll down to the CPUSTRES process, select the second thread (thread 1; the first thread is the GUI thread) and click the **Add** button. You should see something like this:

10. Click **OK**.

11. Right-click the counter and select **Properties**.

12. Click the **Graph** tab and change the maximum vertical scale to 16. Then click **OK**.

13. Bring the CPUSTRES process to the foreground. You should see the priority of the CPUSTRES thread being boosted by 2 and then decaying back to the base priority. CPUSTRES periodically receives a boost of 2 because the thread you're monitoring is sleeping about 25 percent of the time and then waking up. (This is the Busy activity level.) The boost is applied when the thread wakes up. If you set the activity level to Maximum, you won't see any boosts because Maximum in CPUSTRES puts the thread into an infinite loop. Therefore, the thread doesn't invoke any wait functions and therefore doesn't receive any boosts.

14. When you've finished, exit Performance Monitor and CPU Stress.

Priority boosts after GUI threads wake up

Threads that own windows receive an additional boost of 2 when they wake up because of windowing activity such as the arrival of window messages. The windowing system (Win32k.sys) applies this boost when it calls `KeSetEvent` to set an event used to wake up a GUI thread. The reason for this boost is similar to the previous one: to favor interactive applications.

EXPERIMENT: WATCHING PRIORITY BOOSTS ON GUI THREADS

You can see the windowing system apply its boost of 2 for GUI threads that wake up to process window messages by monitoring the current priority of a GUI application and moving the mouse across the window. Just follow these steps:

1. Open the **System** utility in Control Panel.

2. Click the **Advanced System Settings** label, click the **Advanced** tab, click the **Settings** button in the Performance section, and click the **Advanced** tab.

3. Select the **Programs** option. This gives `PsPrioritySeparation` a value of 2.

4. Run Notepad.

5. Start the Performance Monitor tool.

6. Click the **Add Counter** toolbar button or press **Ctrl+I** to open the Add Counters dialog box.

7. Select the **Thread** object and then select the **Priority Current** counter.

8. In the Instances box, type **Notepad**. Then click **Search**.

9. Scroll down to the Notepad/0 entry, click it, click the **Add** button, and then click **OK**.

10. As in the previous experiment, change the maximum vertical scale to 16. You should see the priority of thread 0 in Notepad at 8 or 10. (Because Notepad entered a wait state shortly after it received the boost of 2 that threads in the foreground process receive, it might not yet have decayed from 10 to 8.)

11. With Performance Monitor in the foreground, move the mouse across the Notepad window. (Make both windows visible on the desktop.) Notice that the priority sometimes remains at 10 and sometimes at 9, for the reasons just explained.

NOTE

You won't likely catch Notepad at 8. This is because it runs so little after receiving the GUI thread boost of 2 that it never experiences more than one priority level of decay before waking up again. (This is due to additional windowing activity and the fact that it receives the boost of 2 again.)

---

12. Bring Notepad to the foreground. You should see the priority rise to 12 and remain there. This is because the thread is receiving two boosts: the boost of 2 applied to GUI threads when they wake up to process windowing input and an additional boost of 2 because Notepad is in the foreground. (Or, you may see it drop to 11 if it experiences the normal priority decay that occurs for boosted threads on the quantum end.)

13. Move the mouse over Notepad while it's still in the foreground. You might see the priority drop to 11 (or maybe even 10) as it experiences the priority decay that normally occurs on boosted threads as they complete their turn. However, the boost of 2 that is applied because it's the foreground process remains as long as Notepad remains in the foreground.

14. Exit Performance Monitor and Notepad.

Priority boosts for CPU starvation

Imagine the following situation: A priority 7 thread is running, preventing a priority 4 thread from ever receiving CPU time. However, a priority 11 thread is waiting for some resource that the priority 4 thread has

locked. But because the priority 7 thread in the middle is eating up all the CPU time, the priority 4 thread will never run long enough to finish whatever it's doing and release the resource blocking the priority 11 thread. This scenario is known as *priority inversion*.

What does Windows do to address this situation? An ideal solution (at least in theory) would be to track locks and owners and boost the appropriate threads so that forward progress can be made. This idea is implemented with a feature called *Autoboost*, described later in this chapter in the section "**Autoboost**." However, for general starvation scenarios, the following mitigation is used.

You saw how the code responsible for executive resources manages this scenario by boosting the owner threads so that they can have a chance to run and release the resource. However, executive resources are only one of the many synchronization constructs available to developers, and the boosting technique will not apply to any other primitive. Therefore, Windows also includes a generic CPU starvation-relief mechanism as part of a thread called the *balance-set manager*. (This is a system thread that exists primarily to perform memory-management functions and is described in more detail in **Chapter 5**.) Once per second, this thread scans the ready queues for any threads that have been in the ready state (that is, haven't run) for approximately 4 seconds. If it finds such a thread, the balance-set manager boosts the thread's priority to 15 and sets the quantum target to an equivalent CPU clock cycle count of 3 quantum units. After the quantum expires, the thread's priority decays immediately to its original base priority. If the thread wasn't finished and a higher-priority thread is ready to run, the decayed thread returns to the ready queue, where it again becomes eligible for another boost if it remains there for another 4 seconds.

The balance-set manager doesn't actually scan all the ready threads every time it runs. To minimize the CPU time it uses, it scans only 16 ready threads, If there are more threads at that priority level, it remembers where it left off and picks up again on the next pass. Also, it boosts only 10 threads per pass. If it finds more than 10 threads meriting this

particular boost (which indicates an unusually busy system), it stops the scan and picks up again on the next pass.

NOTE

As mentioned, scheduling decisions in Windows are not affected by the number of threads and are made in constant time. Because the balance-set manager must scan ready queues manually, this operation depends on the number of threads on the system; more threads require more scanning time. However, the balance-set manager is not considered part of the scheduler or its algorithms and is simply an extended mechanism to increase reliability. Additionally, because of the cap on threads and queues to scan, the performance impact is minimized and predictable in a worst-case scenario.

EXPERIMENT: WATCHING PRIORITY BOOSTS FOR CPU STARVATION

Using the CPU Stress tool, you can watch priority boosts in action. In this experiment, you'll see CPU usage change when a thread's priority is boosted. Take the following steps:

1. Run CPUSTRES.exe.

2. The activity level of thread 1 is Low. Change it to **Maximum**.

3. The thread priority of thread 1 is Normal. Change it to **Lowest**.

4. Click thread 2. Its activity level is Low. Change it to **Maximum**.

5. Change the process affinity mask to a single logical processor. To do so, open the **Process** menu and choose **Affinity**. (It doesn't matter which processor.) Alternatively, use Task Manager to make the change. The screen should look something like this:

6. Start the Performance Monitor tool.

7. Click the **Add Counter** toolbar button or press **Ctrl**+**I** to open the Add Counters dialog box.

8. Select the **Thread** object and then select the **Priority Current** counter.

9. In the Instances box, type **CPUSTRES** and click **Search**.

10. Select threads 1 and 2 (thread 0 is the GUI thread), click the **Add** button, and click **OK**.

11. Change the vertical scale maximum to 16 for both counters.

12. Because Performance Monitor refreshes once per second, you may miss the priority boosts. To help with that, press **Ctrl**+**F** to freeze the display. Then force updates to occur more frequently by pressing and holding down **Ctrl**+**U**. With some luck, you may see a priority boost for the lower-priority thread to level 15 like so:

13. Exit Performance Monitor and CPU Stress.